Fix C++26 by making the rank-1, rank-2, rank-k, and rank-2k updates consistent with the BLAS

Document #:	P3371R5
Date:	2025-11-07
Project:	Programming Language C++ LWG

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com> Ilya Burylov <burylov@gmail.com>

Contents

- 1 Authors
- 2 Revision history
- 3 Abstract
- 4 Discussion of proposed changes
  - 4.1 Support both overwriting and updating rank-k and rank-2k updates
    - 4.1.1 For rank-k and rank-2k updates, BLAS supports scaling factor `beta`, while std::linalg currently does not
    - 4.1.2 Make these functions consistent with general matrix product
    - 4.1.3 Fix requires changing behavior of existing overloads
    - 4.1.4 Add new updating overloads; make existing ones overwriting
    - 4.1.5 Summary of proposed changes
  - 4.2 Change rank-1 and rank-2 updates to be consistent with rank-k and rank-2k
    - 4.2.1 Current std::linalg behavior
    - 4.2.2 Current behavior is inconsistent with BLAS Standard and rank-k and rank-2k updates
    - 4.2.3 This change would remove a special case in std::linalg's design
    - 4.2.4 Exposition-only concept no longer needed
    - 4.2.5 Summary of proposed changes

- 4.3 Use only the real part of scaling factor `alpha` for Hermitian matrix rank-1 and rank-k updates
  - 4.3.1 Survey of scaling factors in Hermitian matrix BLAS routines
  - 4.3.2 Translation of Hermitian BLAS concerns to std::linalg
  - 4.3.3 Alternative solutions
  - 4.3.4 Summary of proposed changes
- 4.4 Remove rank-1 and rank-k symmetric and Hermitian update overloads without `alpha`, and constrain `Scalar alpha`
  - 4.4.1 Summary
  - 4.4.2 Motivation
- 4.5 Things relating to scaling factors that we do not propose changing
  - 4.5.1 Hermitian matrix-vector and matrix-matrix products
  - 4.5.2 Triangular matrix products with implicit unit diagonals
  - 4.5.3 Triangular solves with implicit unit diagonals

- 5 Ordering with respect to other proposals and LWG issues
- 6 Implementation status
- 7 Acknowledgments
- 8 Wording
  - 8.1 Constrain all `class Scalar` template parameters
  - 8.2 Change exposition-only concepts
  - 8.3 Rank-1 update functions in synopsis
  - 8.4 Rank-2 update functions in synopsis
  - 8.5 Rank-k update functions in synopsis
  - 8.6 Rank-2k update functions in synopsis
  - 8.7 Constrain linear algebra value type
  - 8.8 Specification of nonsymmetric rank-1 update functions
  - 8.9 Specification of symmetric and Hermitian rank-1 update functions
  - 8.10 Specification of symmetric and Hermitian rank-2 update functions
  - 8.11 Specification of rank-k update functions
  - 8.12 Specification of rank-2k update functions
- 9 Appendix A: Example of a generic numerical algorithm

§ 1 Authors

— Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

— Ilya Burylov (burylov@gmail.com) (NVIDIA)

§ 2 Revision history

— Revision 0 to be submitted 2024-08-15

— Revision 1 to be submitted 2024-09-15

  — Main wording changes from R0:

    — Instead of just changing the symmetric and Hermitian rank-k and rank-2k updates to have both overwriting and updating overloads, change *all* the update functions in this way: rank-1 (general, symmetric, and Hermitian), rank-2, rank-k, and rank-2k

    — For all the new updating overloads, specify that `C` (or `A`) may alias `E`

    — For the new symmetric and Hermitian updating overloads, specify that the functions access the new `E` parameter in the same way (e.g., with respect to the lower or upper triangle) as the `C` (or `A`) parameter

    — Add exposition-only concept *`noncomplex`* to constrain a scaling factor to be noncomplex, as needed for Hermitian rank-1 and rank-k functions

  — Add Ilya Burylov as coauthor

  — Change title and abstract to express the wording changes

  — Add nonwording section explaining why we change rank-1 and rank-2 updates to be consistent with rank-k and rank-2k updates

  — Add nonwording sections explaining why we don't change `hermitian_matrix_vector_product`, `hermitian_matrix_product`, `triangular_matrix_product`, or the triangular solves

  — Add nonwording section explaining why we constrain some scaling factors to be noncomplex at compile time, instead of taking a run-time approach

  — Reorganize and expand nonwording sections

— Revision 2 to be submitted 2024-10-15

  — For Hermitian matrix rank-1 and rank-k updates, do not constrain the type of the scaling factor `alpha`, as R1 did. Instead, define the algorithms to use *`real-if-needed`*`(alpha)`. Remove exposition-only concept *`noncomplex`*.

  — Remove the exposition-only concept *`possibly-packed-in-matrix`* that was introduced in R1, as it is overly restrictive. (Input matrices never need to have

unique layout, even if they are not packed.)

— Revision 3 to be submitted 2024-11-15

  — Remove rank-1 and rank-k update overloads without a `Scalar alpha` parameter. Retain rank-1 and rank-k update overloads with this parameter.

  — Constrain "linear algebra value type" to be neither `mdspan` nor an execution policy (`is_execution_policy_v<Scalar>` is `false`). This will prevent ambiguous overloads, even if we retain overloads without `Scalar alpha` or add them later.

  — Add nonwording sections motivating this change.

— Revision 4 to be submitted 2025-04-06

  — LEWG voted to forward R3 on 2025-03-18.

  — Make wording diff more compact by improving formatting.

  — Update (nonwording) "Implementation status" section.

  — Harmonise wording diff with proposed fix for LWG4137 (atop which this paper was and is rebased) and add Editorial Notes explaining where the rebase affects this proposal's wording changes.

— Revision 5 to be submitted 2025-11-??

  — LEWG voted on 2025-11-04 at the Kona meeting to forward R4 for C++26.

  — LWG reviewed R4 on 2025-11-05; include requested changes.

§ 3 Abstract

We propose the following changes to [linalg] that improve consistency of the rank-1, rank-2, rank-k, and rank-2k update functions with the BLAS.

1. Add "updating" overloads to all the rank-1, rank-2, rank-k, and rank-2k update functions: general, symmetric, and Hermitian. The new overloads are analogous to the updating overloads of `matrix_product`. For example, `symmetric_matrix_rank_k_update(alpha, A, scaled(beta, C), C, upper_triangle)` will perform $C := \beta C + \alpha A A^T$. This makes the functions consistent with the BLAS's behavior for nonzero `beta`, and also more consistent with the behavior of `matrix_product` (of which they are mathematically a special case).

2. Change the behavior of all the existing rank-1, rank-2, rank-k, and rank-2k update functions (general, symmetric, and Hermitian) to be "overwriting" instead of "unconditionally updating." For example,

`symmetric_matrix_rank_k_update(alpha, A, C, upper_triangle)` will perform $C = \alpha AA^T$ instead of $C := C + \alpha AA^T$. This makes them consistent with the BLAS's behavior when `beta` is zero.

3. For the overloads of `hermitian_rank_1_update` and `hermitian_rank_k_update` that have an `alpha` scaling factor parameter, only use *`real-if-needed`*`(alpha)` in the update. This ensures that the update will be mathematically Hermitian, and makes the behavior well defined if `alpha` has nonzero imaginary part. The change is also consistent with our proposed resolution for LWG 4136 ("Specify behavior of [linalg] Hermitian algorithms on diagonal with nonzero imaginary part").

4. Remove overloads of rank-1 and rank-k symmetric and Hermitian update functions without a `Scalar alpha` parameter. Retain only those overloads that have a `Scalar alpha` parameter. In case WG21 wants to add those overloads or similar ones later, constrain linear algebra value types (and therefore `Scalar`) to be neither `mdspan` nor an execution policy. This avoids potentially ambiguous overloads.

Items (2), (3), and (4) are breaking changes to the current Working Draft. Thus, we must finish this before finalization of C++26.

§ 4 Discussion of proposed changes

§ 4.1 Support both overwriting and updating rank-k and rank-2k updates

1. For rank-k and rank-2k updates (general, symmetric, and Hermitian), BLAS routines support both overwriting and updating behavior by exposing a scaling factor `beta`. The corresponding [linalg] algorithms currently do not expose the equivalent functionality. Instead, they are unconditionally updating, as if `beta` is one.

2. The rank-k and rank-2k updates are special cases of `matrix_product`, but as a result of (1), their behavior is not consistent with `matrix_product`.

3. Therefore, we need to add updating overloads of the rank-k and rank-2k updates, and change the existing overloads to be overwriting.

4. The change to existing overloads is a breaking change and thus must be finished before C++26.

5. To simplify wording, we add the new exposition-only concept *`possibly-packed-out-matrix`* for symmetric and Hermitian matrix update algorithms.

§ 4.1.1 For rank-k and rank-2k updates, BLAS supports scaling factor `beta`, while std::linalg currently does not

Each function in any section whose label begins with "linalg.algs" generally corresponds to one or more routines or functions in the original BLAS (Basic Linear Algebra Subroutines). Every computation that the BLAS can do, a function in the C++ Standard Library should be able to do.

One `std::linalg` user reported an exception to this rule. The BLAS routines `xSYRK` (SYmmetric Rank-K update) computes $C := \beta C + \alpha AA^T$, but the corresponding `std::linalg` function `symmetric_matrix_rank_k_update` only computes $C := C + \alpha AA^T$. That is, `std::linalg` currently has no way to express this BLAS operation with a general β scaling factor.

This issue applies to all of the symmetric and Hermitian rank-k and rank-2k update functions. The following table lists these functions and what they compute now. A and B denote general matrices, C denotes a symmetric or Hermitian matrix (depending on the algorithm's name), the superscript `T` denotes the transpose, the superscript `H` denotes the Hermitian transpose, α denotes a scaling factor, and $\bar{\alpha}$ denotes the complex conjugate of α. Making the functions have "updating" overloads that take an input matrix E would permit $E = \beta C$, and thus make [linalg] able to compute what the BLAS can compute.

[linalg] algorithm	**What it computes now**
`symmetric_matrix_rank_k_update`	$C := C + \alpha AA^T$
`hermitian_matrix_rank_k_update`	$C := C + \alpha AA^H$
`symmetric_matrix_rank_2k_update`	$C := C + \alpha AB^T + \alpha BA^T$
`hermitian_matrix_rank_2k_update`	$C := C + \alpha AB^H + \bar{\alpha} BA^H$

§ 4.1.2 Make these functions consistent with general matrix product

These functions implement special cases of matrix-matrix products. The `matrix_product` function in `std::linalg` implements the general case of matrix-matrix products. This function corresponds to the BLAS's `SGEMM`, `DGEMM`, `CGEMM`, and `ZGEMM`, which compute $C := \beta C + \alpha AB$, where α and β are scaling factors. The `matrix_product` function has two kinds of overloads:

1. *overwriting* ($C = AB$) and
2. *updating* ($C = E + AB$).

The updating overloads handle the general α and β case by `matrix_product(scaled(alpha, A), B, scaled(beta, C), C)`. The specification explicitly permits the input `scaled(beta, C)` to alias the output `C` (**[linalg.algs.blas3.gemm]** 10: "*Remarks*: `C` may alias `E`"). The `std::linalg` library provides overwriting and updating overloads so that it can do everything that the BLAS does, just in a more idiomatically C++ way. Please see P1673R13 Section 10.3 ("Function argument aliasing and zero scalar multipliers") for a more detailed explanation.

§ 4.1.3 Fix requires changing behavior of existing overloads

The problem with the current symmetric and Hermitian rank-k and rank-2k functions is that they have the same *calling syntax* as the overwriting version of `matrix_product`, but *semantics* that differ from both the overwriting and the updating versions of `matrix_product`. For example,

```
hermitian_matrix_rank_k_update(alpha, A, C);
```

updates C with $C + \alpha AA^H$, but

```
matrix_product(scaled(alpha, A), conjugate_transposed(A), C);
```

overwrites C with αAA^H. The current rank-k and rank-2k overloads are not overwriting, so we can't just fix this problem by introducing an "updating" overload for each function.

Incidentally, the fact that these functions have "update" in their name is not relevant, because that naming choice is original to the BLAS. The BLAS calls its corresponding `xSYRK`, `xHERK`, `xSYR2K`, and `xHER2K` routines "{Symmetric, Hermitian} rank {one, two} update," even though setting $\beta = 0$ makes these routines "overwriting" in the sense of `std::linalg`.

§ 4.1.4 Add new updating overloads; make existing ones overwriting

We propose to fix this by making the functions work just like `matrix_vector_product` or `matrix_product`. This entails three changes.

1. Add two new exposition-only concept *`possibly-packed-out-matrix`* for constraining output parameters of the changed or new symmetric and Hermitian update functions.

2. Add "updating" overloads of the symmetric and Hermitian rank-k and rank-2k update functions.

    a. The updating overloads take a new input matrix parameter `E`, analogous to the updating overloads of `matrix_product`, and make `C` an output parameter instead of an in/out parameter. For example, `symmetric_matrix_rank_k_update(alpha, A, E, C, upper_triangle)` computes $C = E + \alpha AA^T$.

    b. Explicitly permit `C` and `E` to alias, thus permitting the desired case where `E` is `scaled(beta, C)`.

    c. The updating overloads take `E` as an *`in-matrix`*, and take `C` as a *`possibly-packed-out-matrix`* (instead of a *`possibly-packed-inout-matrix`*).

    d. `E` must be accessed as a symmetric or Hermitian matrix (depending on the function name) and such accesses must use the same triangle as `C`. (The

existing [linalg.general] 4 wording for symmetric and Hermitian behavior does not cover `E`.)

3. Change the behavior of the existing symmetric and Hermitian rank-k and rank-2k overloads to be overwriting instead of updating.

    a. For example, `symmetric_matrix_rank_k_update(alpha, A, C, upper_triangle)` will compute $C = \alpha AA^T$ instead of $C := C + \alpha AA^T$.

    b. Change `C` from a *`possibly-packed-inout-matrix`* to a *`possibly-packed-out-matrix`*.

Items (2) and (3) are breaking changes to the current Working Draft. This needs to be so that we can provide the overwriting behavior $C := \alpha AA^T$ or $C := \alpha AA^H$ that the corresponding BLAS routines already provide. Thus, we must finish this before finalization of C++26.

Both sets of overloads still only write to the specified triangle (lower or upper) of the output matrix `C`. As a result, the new updating overloads only read from that triangle of the input matrix `E`. Therefore, even though `E` may be a different matrix than `C`, the updating overloads do not need an additional `Triangle t_E` parameter for `E`. The `symmetric_*` functions interpret `E` as symmetric in the same way that they interpret `C` as symmetric, and the `hermitian_*` functions interpret `E` as Hermitian in the same way that they interpret `C` as Hermitian. Nevertheless, we do need new wording to explain how the functions may interpret and access `E`.

§ 4.1.5 Summary of proposed changes

[linalg] algorithm	What it computes now	Change (overwriting)	Add (updating)
`symmetric_matrix_rank_k_update`	$C := C + \alpha AA^T$	$C = \alpha AA^T$	$C = E + \alpha AA^T$
`hermitian_matrix_rank_k_update`	$C := C + \alpha AA^H$	$C = \alpha AA^H$	$C = E + \alpha AA^H$
`symmetric_matrix_rank_2k_update`	$C := C + \alpha AB^T + \alpha BA^T$	$C = \alpha AB^T + \alpha BA^T$	$C = E + \alpha AB^T + \alpha BA^T$
`hermitian_matrix_rank_2k_update`	$C := C + \alpha AB^H + \bar{\alpha} BA^H$	$C = \alpha AB^H + \bar{\alpha} BA^H$	$C = E + \alpha AB^H + \bar{\alpha} BA^H$

§ 4.2 Change rank-1 and rank-2 updates to be consistent with rank-k and rank-2k

1. Currently, the rank-1 and rank-2 update functions unconditionally update and do not give users a way to provide a `beta` scaling factor.

2. This behavior deviates from the BLAS Standard, is inconsistent with the rank-k and rank-2k update functions, and introduces a special case in [linalg]'s design.

3. We propose making all the rank-1 and rank-2 update functions consistent with the proposed change to the rank-k and rank-2k update functions. This means both changing the meaning of the current overloads to be overwriting, and adding new overloads that are updating. This includes general (nonsymmetric), symmetric, and Hermitian rank-1 update functions, as well as symmetric and Hermitian rank-2 update functions.

4. The exposition-only concept *`possibly-packed-inout-matrix`* is no longer needed. We propose removing it.

§ 4.2.1 Current std::linalg behavior

The symmetric and Hermitian rank-k and rank-2k update functions have the following rank-1 and rank-2 analogs, A denotes a symmetric or Hermitian matrix (depending on the function's name), x and y denote vectors, and α denotes a scaling factor.

[linalg] algorithm	What it computes now
`symmetric_matrix_rank_1_update`	$A := A + \alpha x x^T$
`hermitian_matrix_rank_1_update`	$A := A + \alpha x x^H$
`symmetric_matrix_rank_2_update`	$A := A + \alpha x y^T + \alpha y x^T$
`hermitian_matrix_rank_2_update`	$A := A + \alpha x y^H + \bar{\alpha} x y^H$

These functions *unconditionally* update the matrix A. They do not have an overwriting option. In this, they follow the "general" (not necessarily symmetric or Hermitian) rank-1 update functions.

[linalg] algorithm	What it computes now
`matrix_rank_1_update`	$A := A + x y^T$
`matrix_rank_1_update_c`	$A := A + x y^H$

§ 4.2.2 Current behavior is inconsistent with BLAS Standard and rank-k and rank-2k updates

These six rank-1 and rank-2 update functions map to BLAS routines as follows.

[linalg] algorithm	Corresponding BLAS routine(s)
`matrix_rank_1_update`	`xGER`
`matrix_rank_1_update_c`	`xGERC`
`symmetric_matrix_rank_1_update`	`xSYR`, `xSPR`

`hermitian_matrix_rank_1_update`	`xHER`, `xHPR`
`symmetric_matrix_rank_2_update`	`xSYR2`, `xSPR2`
`hermitian_matrix_rank_2_update`	`xHER2`, `xHPR2`

The Reference BLAS and the BLAS Standard (see Chapter 2, pp. 64 - 68) differ here. The Reference BLAS and the original 1988 BLAS 2 paper specify all of the rank-1 and rank-2 update routines listed above as unconditionally updating, and not taking a β scaling factor. However, the (2002) BLAS Standard specifies all of these rank-1 and rank-2 update functions as taking a β scaling factor. We consider the latter to express our design intent. It is also consistent with the corresponding rank-k and rank-2k update functions in the BLAS, which all take a β scaling factor and thus can do either overwriting (with zero β) or updating (with nonzero β). These routines include `xSYRK`, `xHERK`, `xSYR2K`, and `xHER2K`. One could also include the general matrix-matrix product `xGEMM` among these, as `xGEMM` also takes a β scaling factor.

§ 4.2.3 This change would remove a special case in std::linalg's design

Section 10.3 of P1673R13 explains the three ways that the std::linalg design translates Fortran `INTENT(INOUT)` arguments into a C++ idiom.

1. Provide both in-place and not-in-place overloads for triangular solve and triangular multiply.

2. "Else, if the BLAS function unconditionally updates (like `xGER`), we retain read-and-write behavior for that argument."

3. "Else, if the BLAS function uses a scalar `beta` argument to decide whether to read the output argument as well as write to it (like `xGEMM`), we provide two versions: a write-only version (as if `beta` is zero), and a read-and-write version (as if `beta` is nonzero)."

Our design goal was for functions with vector or matrix output to imitate `std::transform` as much as possible. This favors Way (3) as the default approach, which turns `INTENT(INOUT)` arguments on the Fortran side into separate input and output parameters on the C++ side. Way (2) is really an awkward special case. The BLAS Standard effectively eliminates this special case by making the rank-1 and rank-2 updates work just like the rank-k and rank-2k updates, with a β scaling factor. This makes it natural to eliminate the Way (2) special case in [linalg] as well.

§ 4.2.4 Exposition-only concept no longer needed

These changes make the exposition-only concept *`possibly-packed-inout-matrix`* superfluous. We propose removing it.

Note that this would not eliminate all uses of the exposition-only concept *`inout-matrix`*. The in-place triangular matrix product functions `triangular_matrix_left_product` and `triangular_matrix_right_product`, and the in-place triangular linear system solve

functions `triangular_matrix_matrix_left_solve` and `triangular_matrix_matrix_right_solve` would still use *`inout-matrix`*.

§ 4.2.5 Summary of proposed changes

[linalg] algorithm	What it computes now	Change (overwriting)	Add (updating)
`matrix_rank_1_update`	$A := A + xy^T$	$A = xy^T$	$A = E + xy^T$
`matrix_rank_1_update_c`	$A := A + xy^H$	$A = xy^H$	$A = E + xy^H$
`symmetric_matrix_rank_1_update`	$A := A + \alpha xx^T$	$A = \alpha xx^T$	$A = E + \alpha xx^T$
`hermitian_matrix_rank_1_update`	$A := A + \alpha xx^H$	$A = \alpha xx^H$	$A = E + \alpha xx^H$
`symmetric_matrix_rank_2_update`	$A := A + \alpha xy^T + \alpha yx^T$	$A = \alpha xy^T + \alpha yx^T$	$A = E + \alpha xy^T + \alpha yx^T$
`hermitian_matrix_rank_2_update`	$A := A + \alpha xy^H + \bar{\alpha} xy^H$	$A = \alpha xy^H + \bar{\alpha} xy^H$	$A = E + \alpha xy^H + \bar{\alpha} xy^H$

§ 4.3 Use only the real part of scaling factor `alpha` for Hermitian matrix rank-1 and rank-k updates

For Hermitian rank-1 and rank-k matrix updates, if users provide a scaling factor `alpha`, it must have zero imaginary part. Otherwise, the matrix update will not be Hermitian, because all elements on the diagonal of a Hermitian matrix must have zero imaginary part. Even though AA^H is mathematically always Hermitian, if α has nonzero imaginary part, then αAA^H may no longer be a Hermitian matrix. For example, if A is the identity matrix (with ones on the diagonal and zeros elsewhere) and $\alpha = i$ (the imaginary unit, which is the square root of negative one), then αAA^H is the diagonal matrix whose diagonal elements are all i, and thus has nonzero imaginary part.

The specification of `hermitian_matrix_rank_1_update` and `hermitian_matrix_rank_k_update` does not currently require that `alpha` have zero imaginary part. We propose fixing this by making these update algorithms only use the real part of `alpha`, as in *`real-if-needed`*`(alpha)`. This solution is consistent with our proposed resolution of LWG Issue 4136, "Specify behavior of [linalg] Hermitian algorithms on diagonal with nonzero imaginary part," where we make Hermitian rank-1 and rank-k matrix updates use only the real part of matrices' diagonals.

We begin with a summary of all the Hermitian matrix BLAS routines, how scaling factors influence their mathematical correctness. Then, we explain how these scaling factor concerns translate into [linalg] function concerns. Finally, we discuss alternative solutions.

§ 4.3.1 Survey of scaling factors in Hermitian matrix BLAS routines

The BLAS's Hermitian matrix routines take `alpha` and `beta` scaling factors. The BLAS addresses the resulting correctness concerns in different ways, depending on what each routine computes. For routines where a nonzero imaginary part could make the result incorrect, the routine restricts the scaling factor to have a noncomplex number type. Otherwise, the routine takes the scaling factor as a complex number type. We discuss all the Hermitian routines here.

§ 4.3.1.1 `xHEMM`: Hermitian matrix-matrix multiply

`xHEMM` (HErmitian Matrix-matrix Multiply) computes either $C := \alpha AB + \beta C$ or $C := \alpha BA + \beta C$, where A is a Hermitian matrix, and neither B nor C need to be Hermitian. The products AB and BA thus need not be Hermitian, so the scaling factors α and β can have nonzero imaginary parts. The BLAS takes them both as complex numbers.

§ 4.3.1.2 `xHEMV`: HErmitian Matrix-Vector multiply

`xHEMV` (HErmitian Matrix-Vector multiply) computes $y := \alpha Ax + \beta y$, where A is a Hermitian matrix and x and y are vectors. The scaled matrix αA does not need to be Hermitian. Thus, α and β can have nonzero imaginary parts. The BLAS takes them both as complex numbers.

§ 4.3.1.3 `xHER`: HErmitian Rank-1 update

`xHER` (HErmitian Rank-1 update) differs between the Reference BLAS (which computes $A := \alpha xx^H + A$) and the BLAS Standard (which computes $A := \alpha xx^H + \beta A$). The matrix A must be Hermitian, and the rank-1 matrix xx^H is always mathematically Hermitian, so both α and β need to have zero imaginary part in order for the update to preserve A's Hermitian property. The BLAS takes them both as real (noncomplex) numbers.

§ 4.3.1.4 `xHER2`: HErmitian Rank-2 update

`xHER2` (HErmitian Rank-2 update) differs between the Reference BLAS (which computes $A := \alpha xy^H + \bar{\alpha} yx^H + A$, where $\bar{\alpha}$ denotes the complex conjugate of α) and the BLAS Standard (which computes $A := \alpha xy^H + \bar{\alpha} yx^H + \beta A$). The matrix A must be Hermitian, and the rank-2 matrix $\alpha xy^H + \bar{\alpha} yx^H$ is always mathematically Hermitian, no matter the value of α. Thus, α can have nonzero imaginary part, but β cannot. The BLAS thus takes `alpha` as a complex number, but `beta` as a real (noncomplex) number. (There is likely a typo in the BLAS Standard's description of the Fortran 95 binding. It says that both `alpha` and `beta` are complex (have type `COMPLEX(<wp>)`), even though in the Fortran 77 binding,

`beta` is real (`<rtype>`). The BLAS Standard's description of `xHER2K` (see below) says that `alpha` is complex but `beta` is real. `xHER2` needs to be consistent with `xHER2K`.)

§ 4.3.1.5 `xHERK`: HErmitian Rank-K update

`xHERK` (HErmitian Rank-K update) computes either $C := \alpha A A^H + \beta C$ or $C := \alpha A^H A + \beta C$, where C must be Hermitian. This is a generalization of `xHER` and thus both α and β need to have zero imaginary part. The BLAS takes them both as real (noncomplex) numbers.

§ 4.3.1.6 `xHER2K`: HErmitian Rank-2k update

`xHER2K` (HErmitian Rank-2k update) computes either $C := \alpha A B^H + \bar{\alpha} B A^H + \beta C$ or $C := \alpha A^H B + \bar{\alpha} B^H A + \beta C$. This is a generalization of `xHER2`: α can have nonzero imaginary part, but β cannot. The BLAS thus takes `alpha` as a complex number, but `beta` as a real (noncomplex) number.

§ 4.3.1.7 Summary of BLAS routine restrictions

The following table lists, for all the Hermitian matrix update BLAS routines, whether the routine restricts `alpha` and/or `beta` to have zero imaginary part, and whether the routine is a generalization of some other routine in the list (N/A, "not applicable," means that it is not).

BLAS routine	Restricts `alpha`	Restricts `beta`	Generalizes
`xHEMM`	No	No	N/A
`xHER`	Yes	Yes	N/A
`xHER2`	No	Yes	N/A
`xHERK`	Yes	Yes	`xHER`
`xHER2K`	No	Yes	`xHER2`

§ 4.3.2 Translation of Hermitian BLAS concerns to std::linalg

§ 4.3.2.1 Assume changes proposed in previous sections

We assume here the changes proposed in previous sections that remove inout matrix parameters from the rank-1, rank-2, rank-k, and rank-2k algorithms, and separate these algorithms into overwriting and updating overloads. This lets us only consider input matrix and vector parameters.

§ 4.3.2.2 std::linalg and the BLAS treat scaling factors differently

The [linalg] library and the BLAS treat scaling factors in different ways. First, [linalg] treats the result of `scaled` just like any other matrix or vector parameter. It applies any mathematical requirements (like being Hermitian) to the parameter, regardless of whether the corresponding argument results from `scaled`. It also does not forbid any input argument from being the result of `scaled`. Second, the BLAS always exposes `alpha` and

`beta` scaling factor parameters separately from the matrix or vector parameters to which they are applied. In contrast, [linalg] only exposes a separate `alpha` scaling factor (never `beta`) if it would otherwise be mathematically impossible to express an operation that the BLAS can express. For example, for matrices and scaling factors that are noncomplex, `symmetric_matrix_rank_1_update` cannot express $A := A - xx^T$ by applying `scaled` to `x` with a noncomplex scaling factor (because the square root of -1 is i).

§ 4.3.2.3 Defer fixing places where the BLAS can do what std::linalg cannot

In some cases, [linalg] does not expose a separate scaling factor parameter, even when this prevents [linalg] from doing some things that the BLAS can do. We give an example below of triangular matrix solves with multiple right-hand sides and `alpha` scaling not equal to one, where the matrix has an implicit unit diagonal.

Even though this means that the BLAS can do some things that [linalg] cannot do, it does not cause [linalg] to violate mathematical consistency. More importantly, as we show later, [linalg] can be extended to do whatever the BLAS can do without breaking backwards compatibility. Thus, we consider the status quo acceptable for C++26.

§ 4.3.2.4 Scaling factor `beta` is not a concern

The [linalg] library never exposes a scaling factor `beta`. For BLAS routines that perform an update with `beta` times an inout matrix or vector parameter (e.g., βy or βC), [linalg] instead takes an input matrix or vector parameter (e.g., `E`) that can be separate from the output matrix or vector (e.g., `C`). For Hermitian BLAS routines where `beta` needs to have zero imaginary part, [linalg] simply requires that `E` be Hermitian – a strictly more general requirement. For example, for the new updating overloads of `hermitian_rank_1_update` and `hermitian_rank_k_update` proposed above, [linalg] expresses a `beta` scaling factor by letting users supply `scaled(beta, C)` as the argument for `E`. The wording only requires that `E` be Hermitian. If `E` is `scaled(beta, C)`, this concerns only the product of `beta` and `C`. It would be incorrect to constrain `beta` or `C` separately. For example, if $\beta = -i$ and C is the matrix whose elements are all i, then C is not Hermitian but βC (and therefore `scaled(beta, C)`) is Hermitian. The same reasoning applies for the rank-2 and rank-2k updates.

§ 4.3.2.5 What this section proposes to fix, and the proposed solution

The above arguments help us restrict our concerns. This section of our proposal concerns itself with Hermitian matrix update algorithms where

— the algorithm exposes a separate scaling factor parameter `alpha`, and

— `alpha` needs to have zero imaginary part, but

— nothing in the wording currently prevents `alpha` from having nonzero imaginary part.

These correspond exactly to the BLAS's Hermitian matrix update routines where the type of `alpha` is real: `xHER` and `xHERK`. This strongly suggests solving the problem in [linalg] by constraining the type of `alpha` to be noncomplex. However, as we explain in "Alternative solutions" below, it is hard to define a "noncomplex number" constraint that works well for user-defined number types. Instead, we propose fixing this in a way that is consistent with our proposed resolution of LWG Issue 4136, "Specify behavior of [linalg] Hermitian algorithms on diagonal with nonzero imaginary part." That is, the Hermitian rank-1 and rank-k update algorithms will simply use *`real-if-needed`*`(alpha)` and ignore any nonzero imaginary part of `alpha`.

§ 4.3.3 Alternative solutions

We can think of at least four different ways to solve this problem, and will explain why we did not choose those solutions.

1. Constrain `alpha` by defining a generic "noncomplex number type" constraint.

2. Only constrain `alpha` not to be `std::complex<T>`; do not try to define a generic "noncomplex number" constraint.

3. Constrain `alpha` by default using a generic "noncomplex number type" constraint, but let users specialize an "opt-out trait" to tell the library that their number type is noncomplex.

4. Impose a precondition that the imaginary part of `alpha` is zero.

If we had to pick one of these solutions, we would favor first (4), then (3), and then (1). We would object most to (2).

§ 4.3.3.1 Alternative: Constrain `alpha` via generic "noncomplex" constraint

The BLAS solves this problem by having the Hermitian rank-1 update routines `xHER` and rank-k update routines `xHERK` take the scaling factor α as a noncomplex number. This suggests constraining `alpha`'s type to be noncomplex. However, [linalg] accepts user-defined number types, including user-defined complex number types. How do we define a "noncomplex number type"? If we get it wrong and say that a number type is complex when its "imaginary part" is always zero, we end up excluding a perfectly valid custom number type.

For number types that have an additive identity (like zero for the integers and reals), it's mathematically correct to treat those as "complex numbers" whose imaginary parts are always zero. This is what `conjugated_accessor` does if you give it a number type for which ADL cannot find `conj`. P3050 optimizes `conjugated` for such number types by bypassing `conjugated_accessor` and using `default_accessor` instead, but this is just a code optimization. Therefore, it can afford to be conservative: if a number type *might* be complex, then `conjugated` needs to use `conjugated_accessor`. This is why P3050's approach doesn't apply here. If a number type has ADL-findable `conj`, `real`, and `imag`,

then it *might* be complex. However, if we define the opposite of that as "noncomplex," then we might be preventing users from using otherwise reasonable number types.

The following `MyRealNumber` example always has zero imaginary part, but nevertheless has ADL-findable `conj`, `real`, and `imag`. Furthermore, it has a constructor for which `MyRealNumber(1.2, 3.4)` is well-formed. (This is an unfortunate design choice; making `precision` have class type that is not implicitly convertible from `double` would be wiser, so that users would have to type something like `MyRealNumber(1.2, Precision(42))`.) As a result, there is no reasonable way to tell at compile time if `MyRealNumber` might represent a complex number.

```
class MyRealNumber {
public:
  explicit MyRealNumber(double initial_value);
  // precision represents the amount of storage
  // used by an arbitrary-precision real number object
  explicit MyRealNumber(double initial_value, int precision);

  MyRealNumber(); // result is the additive identity "zero"

  // ... other members to make MyRealNumber regular ...
  // ... hidden friend overloaded arithmetic operators ...

  friend MyRealNumber conj(MyRealNumber x) { return  x; }
  friend MyRealNumber real(MyRealNumber x) { return  x; }
  friend MyRealNumber imag(MyRealNumber)   { return {}; }
};
```

It's reasonable to write custom noncomplex number types that define ADL-findable `conj`, `real`, and `imag`. First, users may want to write or use libraries of generic numerical algorithms that work for both complex and noncomplex number types. P1673 argues that defining `conj` to be type-preserving (unlike `std::conj` in the C++ Standard Library) makes this possible. For example, Appendix A below shows how to implement a generic two-norm absolute value (or magnitude, for complex numbers) function, using this interface. Second, the Standard Library might lead users to write a type like this. `std::conj` accepts arguments of any integer or floating-point type, none of which represent complex numbers. The Standard makes the unfortunate choice for `std::conj` of an integer type to return `std::complex<double>`. However, users who try to make `conj(MyRealNumber)` return `std::complex<MyRealNumber>` would find out that `std::complex<MyRealNumber>` does not compile, because `std::complex<T>` requires that `T` be a floating-point type. The least-effort next step would be to make `conj(MyRealNumber)` return `MyRealNumber`.

We want rank-1 and rank-k Hermitian matrix updates to work with types like `MyRealNumber`, but any reasonable way to constrain the type of `alpha` would exclude `MyRealNumber`.

§ 4.3.3.2 Alternative: Only constrain `alpha` not to be `std::complex`

A variant of this suggestion would be only to constrain `alpha` not to be `std::complex<T>`, and not try to define a generic "noncomplex number" constraint. However, this would break generic numerical algorithms by making valid code for the non-Standard complex number case invalid code for `std::complex<T>`. We do not want [linalg] to treat custom complex number types differently than `std::complex`.

§ 4.3.3.3 Alternative: Constrain `alpha` by default, but let users "opt out"

Another option would be to constrain `alpha` by default using the same generic "noncomplex number type" constraint as in Option (1), but let users specialize an "opt-out trait" to tell the library that their number type is noncomplex. We would add a new "tag" type trait `is_noncomplex` that users can specialize. By default, `is_noncomplex<T>::value` is `false` for any type `T`. This does *not* mean that the type is complex, just that the user declares their type to be noncomplex. The distinction matters, because a noncomplex number type might still provide ADL-findable `conj`, `real`, and `imag`, as we showed above. Users must take positive action to declare their type `U` as "not a complex number type," by specializing `is_noncomplex<U>` so that `is_noncomplex<U>::value` is `true`. If users do that, then the library will ignore any ADL-findable functions `conj`, `real`, and `imag` (whether or not they exist), and will assume that the number type is noncomplex.

Standard Library precedent for this approach is in heterogeneous lookup for associative containers (see N3657 for ordered associative containers, and P0919 and P1690 for unordered containers). User-defined hash functions and key equality comparison functions can tell the container to provide heterogeneous comparisons by exposing a `static constexpr bool is_transparent` whose value is `true`. Default behavior does not expose heterogeneous comparisons. Thus, users must opt in at compile time to assert something about their user-defined types. Another example is `uses_allocator<T, alloc>`, whose `value` member defaults to `false` unless `T` has a nested type `allocator_type` that is convertible from `Alloc`. Standard Library types like `tuple` use `uses_allocator` to determine if a user-defined type `T` is allocator-aware.

Of the three constraint-based approaches discussed in this proposal, we favor this one the most. It still treats types "as they are" and does not permit users to claim that a type is complex when it lacks the needed operations, but it lets users optimize by giving the Standard Library a single bit of compile-time information. By default, any linear algebra value type (see [linalg.reqs.val]) that meets the `maybe_complex` concept below would be considered "possibly complex." Types that do not meet this concept would result in compilation errors; users would then be able to search documentation or web pages to find out that they need to specialize `is_noncomplex`.

```
template<class T>
concept maybe_complex =
  std::semiregular<T> &&
  requires(T t) {
    {conj(t)} -> T;
    {real(t)} -> std::convertible_to<T>;
```

```
    {imag(t)} -> std::convertible_to<T>;
  };
```

P1673 generally avoids approaches based on specializing traits. Its design philosophy favors treating types as they are. Users should not need to do something "extra" with their custom number types to get correct behavior, beyond what they would reasonably need to define to make a custom number type behave like a number.

We base this principle on our past experiences in generic numerical algorithms development. In the 2010's, one of the authors maintained a generic mathematical algorithms library called Trilinos. The Teuchos (pronounced "TEFF-os") package of Trilinos provides a monolithic `ScalarTraits` class template that defines different properties of a number type. It combines the features of `std::numeric_limits` with generic complex arithmetic operations like `conjugate`, `real`, and `imag`. Trilinos' generic algorithms assume that number types are regular and define overloaded `+`, `-`, `*`, and `/`, but use `ScalarTraits<T>::conjugate`, `ScalarTraits<T>::real`, and `ScalarTraits<T>::imag`. As a result, users with a custom complex number type had to specialize `ScalarTraits` and provide all these operations. Even if users had imitated `std::complex`'s interface perfectly and provided ADL-findable `conj`, `real`, and `imag`, users had to do extra work to make Trilinos compile and run correctly for their numbers. With P1673, we decided instead that users who define a custom complex number type with an interface sufficiently like `std::complex` should get reasonable behavior without needing to do anything else.

As a tangent, we would like to comment on the monolithic design of `Teuchos::ScalarTraits`. The monolithic design was partly an imitation of `std::numeric_limits`, and partly a side effect of a requirement to support pre-C++11 compilers that did not permit partial specialization of function templates. (The typical pre-C++11 work-around is to define an unspecialized function template that dispatches to a possibly specialized class template.) Partial specialization of function templates and C++14's variable templates both encourage "breaking up" monolithic traits classes into separate traits. Our paper P1370R1 ("Generic numerical algorithm development with(out) `numeric_limits`") aligns with this trend.

§ 4.3.3.4 Alternative: Impose precondition on `alpha`

Another option would be to impose a precondition that *`imag_if_needed`*`(alpha)` is zero. However, this would be inconsistent with our proposed resolution of LWG Issue 4136, "Specify behavior of [linalg] Hermitian algorithms on diagonal with nonzero imaginary part". WG21 members have expressed wanting *fewer* preconditions and *less* undefined behavior in the Standard Library.

If users call Hermitian matrix rank-1 or rank-k updates with `alpha` being `std::complex<float>` or `std::complex<double>`, implementations of [linalg] that call an underlying C or Fortran BLAS would have to get the real part of `alpha` anyway, because these BLAS routines only take `alpha` as a real type. Thus, our proposed solution –

to *define* the behavior of the update algorithms as using *`real-if-needed`*`(alpha)` – would not add overhead.

§ 4.3.4 Summary of proposed changes

For Hermitian matrix update algorithms where

— the algorithm exposes a separate scaling factor parameter `alpha`, and

— `alpha` needs to have zero imaginary part, but

— nothing in the wording currently prevents `alpha` from having nonzero imaginary part,

specify that these algorithms use *`real-if-needed`*`(alpha)` and ignore any nonzero imaginary part of `alpha`.

§ 4.4 Remove rank-1 and rank-k symmetric and Hermitian update overloads without `alpha`, and constrain `Scalar alpha`

§ 4.4.1 Summary

For the rank-1 and rank-k symmetric and Hermitian update functions, Revision 3 of this paper adds two changes.

1. Constrain linear algebra value types (and thus `Scalar alpha`) to be neither `mdspan` nor an execution policy.

2. Remove overloads that do not have a `Scalar alpha` parameter. Keep the overloads that have a `Scalar alpha` parameter.

§ 4.4.2 Motivation

§ 4.4.2.1 Constraining `Scalar` prevents ambiguous overloads

As motivation for constraining `Scalar`, consider `symmetric_matrix_rank_k_update`. Previous revisions of this paper added the first overload (updating, taking `E` as well as `C`).

```
template<in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  InMat1 A,
  InMat2 E,
  OutMat C,
  Triangle t);

template<class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
```

```
          class Triangle>
void symmetric_matrix_rank_k_update(
  Scalar alpha,
  InMat A,
  OutMat C,
  Triangle t);
```

Our implementation experiments showed that for `mdspan` `A` and `C`, the call `symmetric_matrix_rank_k_update(A, C, C, Triangle{})` is ambiguous. This is because the `Scalar` template parameter is not sufficiently constrained, so it could match either an `mdspan` (as in the first overload above) or `Scalar alpha` (as in the second overload above).

Throughout **[linalg]**, any template parameter named `Scalar` is constrained to be a "linear algebra value type." The definition in **[linalg.reqs.val]** 1 – 3 currently only constrains linear algebra value types to be `semiregular`. Constraining it further to be neither `mdspan` nor an execution policy resolves this ambiguity.

Another option would be to constrain `Scalar` to be multipliable by something. However, this would go against the wording style expressed in **[linalg.reqs.alg]**. Instead of constraining types, that section merely says, "It is a precondition of the algorithm that [any mathematical expression that the algorithm might reasonably use] is a well-formed expression." That is, the algorithms generally don't constrain linear algebra value types to meet their expression requirements. This imitates the wording of Standard Algorithms like `accumulate` and `reduce`.

§ 4.4.2.2 Removing non-`Scalar` overloads simplifies wording and implementations

We propose to go further. For any algorithm that needs an `alpha` parameter overload in order to make sense, we propose discarding the overloads that do *not* have an `alpha` parameter, and keeping only the overloads that do. The only algorithms that would be affected are the symmetric and Hermitian rank-1 and rank-k updates.

This change halves the number of overloads of the symmetric and Hermitian rank-1 and rank-k functions. This mitigates the addition of updating overloads. Retaining only the `alpha` overloads also makes correct use of this interface more obvious. For example, if users want to perform a symmetric rank-k update $C = C + \alpha AA^T$, they would have to write it like this.

```
symmetric_matrix_rank_k_update(alpha, A, C, C, Triangle{});
```

Users no longer would be able to write code like the following, which would scale by α^2 instead of just α and thus would not express what the user meant.

```
symmetric_matrix_rank_k_update(scaled(alpha, A), C, C, Triangle{});
```

In terms of functionality, the only reason to retain non-`alpha` overloads would be to support matrix and vector element types that lack a multiplicative identity. However, the C

or Fortran BLAS does not support such types now, and personal experience with generic C++ linear algebra libraries is that users have never asked for such types. Removing support for such types from the symmetric and Hermitian rank-1 and rank-k updates would reduce the testing burden. Furthermore, we could always add the overloads back later, and we propose constraining the type of `alpha` in a way that makes such overloads not ambiguous.

In terms of performance, one argument for retaining `alpha` overloads is to speed up the special case of `alpha = 1`, by avoiding unnecessary multiplies with `alpha`. Performance of arithmetic operations is more important for "BLAS 3" operations like rank-k updates than for "BLAS 2" operations like rank-1 updates, so we will only consider rank-k updates here. The high-performance case of rank-k updates would likely dispatch to a BLAS or other optimized library that dispatches at run time based on special cases of `alpha`. Thus, there's no need to expose `alpha = 1` as a special case in the interface. Furthermore, the case `alpha = -1` is also an important special case where implementations could avoid multiplications, yet **[linalg]** does not have special interfaces for `alpha = -1`. Thus, we see no pressing motivation to provide special interfaces for the case `alpha = 1`, either.

§ 4.5 Things relating to scaling factors that we do not propose changing

§ 4.5.1 Hermitian matrix-vector and matrix-matrix products

Both `hermitian_matrix_vector_product` and `hermitian_matrix_product` expect that the (possibly scaled) input matrix is Hermitian, while the corresponding BLAS routines `xHEMV` and `xHEMM` expect that the unscaled input matrix is Hermitian and permit the scaling factor `alpha` to have nonzero imaginary part. However, this does not affect the ability of these [linalg] algorithms to compute what the BLAS can compute. Users who want to supply `alpha` with nonzero imaginary part should *not* scale the matrix `A` (as in `scaled(alpha, A)`). Instead, they should scale the input vector `x`, as in the following.

```
auto alpha = std::complex{0.0, 1.0};
hermitian_matrix_vector_product(A, upper_triangle, scaled(alpha, x), y);
```

Therefore, `hermitian_matrix_vector_product` and `hermitian_matrix_product` do *not* need extra overloads with a scaling factor `alpha` parameter.

§ 4.5.1.1 In BLAS, matrix is Hermitian, but scaled matrix need not be

In Chapter 2 of the BLAS Standard, both `xHEMV` and `xHEMM` take the scaling factors α and β as complex numbers (`COMPLEX<wp>`, where `<wp>` represents the current working precision). The BLAS permits `xHEMV` or `xHEMM` to be called with `alpha` whose imaginary part is nonzero. The matrix that the BLAS assumes to be Hermitian is A, not αA. Even if A is Hermitian, αA might not necessarily be Hermitian. For example, if A is the identity matrix (diagonal all ones) and α is i, then αA is not Hermitian but skew-Hermitian.

The current [linalg] wording requires that the input matrix be Hermitian. This excludes replicating BLAS behavior by using `scaled(alpha, A)` (where `alpha` has nonzero imaginary part, and `A` is any Hermitian matrix) as the input matrix. Note that the behavior of this is still otherwise well defined, at least after applying the fix proposed in LWG4136 for diagonal elements with nonzero imaginary part. It does not violate a precondition. Therefore, the Standard has no way to tell the user that they did something wrong.

§ 4.5.1.2 Status quo permits scaling via the input vector

The current wording permits introducing the scaling factor `alpha` through the input vector, even if `alpha` has nonzero imaginary part.

```
auto alpha = std::complex{0.0, 1.0};
hermitian_matrix_vector_product(A, upper_triangle, scaled(alpha, x), y);
```

This is mathematically correct as long as αAx equals $A\alpha x$, that is, as long as α commutes with the elements of A. This issue would only be of concern if those multiplications might be noncommutative. Multiplication with floating-point numbers, integers, and anything that behaves reasonably like a real or complex number is commutative. However, practical number types exist that have noncommutative multiplication. Quaternions are one example. Another is the ring of square N by N matrices (for some fixed dimension N), with matrix multiplication using the same definition that `linalg::matrix_product` uses. One way for a user-defined complex number type to have noncommutative multiplication would be if its real and imaginary parts each have a user-defined number type with noncommutative multiplication, as in the `user_complex<noncommutative>` example below.

```
template<class T>
class user_complex {
public:
  user_complex(T re, T im) : re_(re), im_(im) {}

  friend T real(user_complex<T> z) { return z.re_; }
  friend T imag(user_complex<T> z) { return z.im_; }
  friend user_complex<T> conj(user_complex<T> z) {
    return {real(z), -imag(z)};
  }

  // ... other overloaded arithmetic operators ...

  // the usual complex arithmetic definition
  friend user_complex<T>
  operator*(user_complex<T> z, user_complex<T> w) {
    return {
      real(z) * real(w) - imag(z) * imag(w),
      real(z) * imag(w) + imag(z) * real(w)
    };
  }

private:
  T re_, im_;
```

```
};

class noncommutative {
public:
  explicit noncommutative(double value) : value_(value) {}

  // ... overloaded arithmetic operators ...

  // x * y != y * x here, for example with x=3 and y=5
  friend auto operator*(noncommutative x, noncommutative y) {
    return x + 2.0 * y.value_;
  }

private:
  double value_;
};

auto alpha = user_complex<noncommutative>{
  noncommutative{3.0}, noncommutative{4.0}
};
hermitian_matrix_vector_product(N, upper_triangle, scaled(alpha, x), y);
```

The [linalg] library was designed to support element types with noncommutative multiplication. On the other hand, generally, if we speak of Hermitian matrices or even of inner products (which are used to define Hermitian matrices), we're working in a vector space. This means that multiplication of the matrix's elements is commutative. Thus, we think it is not so onerous to restrict use of `alpha` with nonzero imaginary part in this case.

§ 4.5.1.3 Scaling via the input vector is the least bad choice

Many users may not like the status quo of needing to scale `x` instead of `A`. First, it differs from the BLAS, which puts `alpha` before `A` in its `xHEMV` and `xHEMM` function arguments. Second, users would get no compile-time feedback and likely no run-time feedback if they scale `A` instead of `x`. Their code would compile and produce correct results for almost all matrix-vector or matrix-matrix products, *except* for the Hermitian case, and *only* when the scaling factor has a nonzero imaginary part. However, we still think the status quo is the least bad choice. We explain why by discussing the following alternatives.

1. Treat `scaled(alpha, A)` as a special case: expect `A` to be Hermitian and permit `alpha` to have nonzero imaginary part

2. Forbid `scaled(alpha, A)` at compile time, so that users must scale `x` instead

3. Add overloads that take `alpha`, analogous to the rank-1 and rank-k update functions

The first choice is mathematically incorrect, as we will explain below. The second is not incorrect, but could only catch some errors. The third is likewise not incorrect, but would add a lot of overloads for an uncommon use case, and would still not prevent users from scaling the matrix in mathematically incorrect ways.

4.5.1.3.1 Treating a scaled matrix as a special case would be incorrect

"Treat `scaled(alpha, A)` as a special case" actually means three special cases, given some nested accessor type `Acc` and a scaling factor `alpha` of type `Scalar`.

a. `scaled(alpha, A)`, whose accessor type is `scaled_accessor<Scalar, Acc>`

b. `conjugated(scaled(alpha, A))`, whose accessor type is `conjugated_accessor<scaled_accessor<Scalar, Acc>>`

c. `scaled(alpha, conjugated(A))`, whose accessor type is `scaled_accessor<Scalar, conjugated_accessor<Acc>>`

One could replace `conjugated` with `conjugate_transposed` (which we expect to be more common in practice) without changing the accessor type.

This approach violates the fundamental [linalg] principle that "… each `mdspan` parameter of a function behaves as itself and is not otherwise 'modified' by other parameters" (Section 10.2.5, P1673R13). The behavior of [linalg] is agnostic of specific accessor types, even though implementations likely have optimizations for specific accessor types. [linalg] takes this approach for consistency, even where it results in different behavior from the BLAS (see Section 10.5.2 of P1673R13). The application of this principle here is "the input parameter `A` is always Hermitian." In this case, the consistency matters for mathematical correctness. What if `scaled(alpha, A)` is Hermitian, but `A` itself is not? An example is $\alpha = -i$ and A is the 2 x 2 matrix whose elements are all i. If we treat `scaled_accessor` as a special case, then `hermitian_matrix_vector_product` will compute different results.

Another problem with this approach is that users might define their own accessor types with the effect of `scaled_accessor`, or combine existing nested accessor types in hard-to-detect ways (like a long nesting of `conjugated_accessor` with a `scaled_accessor` inside). The [linalg] library has no way to detect all possible ways that the matrix might be scaled.

4.5.1.3.2 Forbidding `scaled_accessor` would not solve the problem

Hermitian matrix-matrix and matrix-vector product functions could instead *forbid* scaling the matrix at compile time. This means excluding, at compile time, the three accessor type cases listed in the previous option.

a. `scaled_accessor<Scalar, Acc>`

b. `conjugated_accessor<scaled_accessor<Scalar, Acc>>`

c. `scaled_accessor<Scalar, conjugated_accessor<Acc>>`

Doing this would certainly encourage correct behavior for the most common cases. However, as mentioned above, users are permitted to define their own accessor types, and to combine existing nested accessors in arbitrary ways. The [linalg] library cannot detect all possible ways that an arbitrary, possibly user-defined accessor might scale the matrix.

Furthermore, scaling the matrix might still be mathematically correct. A scaling factor with nonzero imaginary part might even *make* the matrix Hermitian. Applying i as a scaling factor twice would give a perfectly valid scaling factor of -1.

4.5.1.3.3 Adding `alpha` overloads would make the problem worse

One could imagine adding overloads that take `alpha`, analogous to the rank-1 and rank-k update overloads that take `alpha`. Users could then write

```
hermitian_matrix_vector_product(alpha, A, upper_triangle, x, y);
```

instead of

```
hermitian_matrix_vector_product(A, upper_triangle, scaled(alpha, x), y);
```

We do not support this approach. First, users can already introduce a scaling factor through the input vector parameter, so adding `alpha` overloads would not add much to what the existing overloads can accomplish. Contrast this with the rank-1 and rank-k Hermitian update functions, where not having `alpha` overloads would prevent simple cases, like $C := C - 2xx^H$ with a user-defined complex number type whose real and imaginary parts are both integers. Second, `alpha` overloads would not prevent users from *also* supplying `scaled(gamma, A)` as the matrix for some other scaling factor `gamma`. This would make the problem worse, because users would need to reason about the combination of two ways that the matrix could be scaled.

4.5.2 Triangular matrix products with implicit unit diagonals

1. In BLAS, triangular matrix-vector and matrix-matrix products apply `alpha` scaling to the implicit unit diagonal. In [linalg], the scaling factor `alpha` is not applied to the implicit unit diagonal. This is because the library does not interpret `scaled(alpha, A)` differently than any other `mdspan`.

2. Users of triangular matrix-vector products can recover BLAS functionality by scaling the input vector instead of the input matrix, so this only matters for triangular matrix-matrix products.

3. All calls of the BLAS's triangular matrix-matrix product routine `xTRMM` in LAPACK (other than in testing routines) use `alpha` equal to one.

4. Straightforward approaches for fixing this issue would not break backwards compatibility.

5. Therefore, we do not consider fixing this a high-priority issue, and we do not propose a fix for it in this paper.

4.5.2.1 BLAS applies alpha after unit diagonal; linalg applies it before

The `triangular_matrix_vector_product` and `triangular_matrix_product` algorithms have an `implicit_unit_diagonal` option. This makes the algorithm not access the diagonal of the matrix, and compute as if the diagonal were all ones. The option corresponds to the BLAS's "Unit" flag. BLAS routines that take both a "Unit" flag and an `alpha` scaling factor apply "Unit" *before* scaling by `alpha`, so that the matrix is treated as if it has a diagonal of all `alpha` values. In contrast, [linalg] follows the general principle that `scaled(alpha, A)` should be treated like any other kind of `mdspan`. As a result, algorithms interpret `implicit_unit_diagonal` as applied to the matrix *after* scaling by `alpha`, so that the matrix still has a diagonal of all ones.

§ 4.5.2.2 Triangular solve algorithms not affected

The triangular solve algorithms in std::linalg are not affected, because their BLAS analogs either do not take an `alpha` argument (as with `xTRSV`), or the `alpha` argument does not affect the triangular matrix (with `xTRSM`, `alpha` affects the right-hand sides `B`, not the triangular matrix `A`).

§ 4.5.2.3 Triangular matrix-vector product work-around

This issue only reduces functionality of `triangular_matrix_product`. Users of `triangular_matrix_vector_product` who wish to replicate the original BLAS functionality can scale the input matrix (by supplying `scaled(alpha, x)` instead of `x` as the input argument) instead of the triangular matrix.

§ 4.5.2.4 Triangular matrix-matrix product example

The following example computes $A := 2AB$ where A is a lower triangular matrix, but it makes the diagonal of A all ones on the input (right-hand) side.

```
triangular_matrix_product(scaled(2.0, A), lower_triangle,
        implicit_unit_diagonal, B, A);
```

Contrast with the analogous BLAS routine `DTRMM`, which has the effect of making the diagonal elements all `2.0`.

```
dtrmm('Left side', 'Lower triangular', 'No transpose', 'Unit diagonal', m,
        n, 2.0, A, lda, B, ldb)
```

If we want to use [linalg] to express what the BLAS call expresses, we need to perform a separate scaling.

```
triangular_matrix_product(A, lower_triangle, implicit_unit_diagonal, B,
        A);
scale(2.0, A);
```

This is counterintuitive, and may also affect performance. Performance of `scale` is typically bound by memory bandwidth and/or latency, but if the work done by `scale` could be fused with the work done by the `triangular_matrix_product`, then `scale`'s memory operations could be "hidden" in the cost of the matrix product.

§ 4.5.2.5 LAPACK never calls `xTRMM` with the implicit unit diagonal option and `alpha` not equal to one

How much might users care about this missing [linalg] feature? P1673R13 explains that the BLAS was codesigned with LAPACK and that every reference BLAS routine is used by some LAPACK routine. "The BLAS does not aim to provide a complete set of mathematical operations. Every function in the BLAS exists because some LINPACK or LAPACK algorithm needs it" (Section 10.6.1). Therefore, to judge the urgency of adding new functionality to [linalg], we can ask whether the functionality would be needed by a C++ re-implementation of LAPACK. We think not much, because the highest-priority target audience of the BLAS is LAPACK developers, and LAPACK routines (other than testing routines) never use a scaling factor alpha other than one.

We survey calls to `xTRMM` in the latest version of LAPACK as of the publication date of R1 of this proposal, LAPACK 3.12.0. It suffices to survey `DTRMM`, the double-precision real case, since for all the routines of interest, the complex case follows the same pattern. (We did survey `ZTRMM`, the double-precision complex case, just in case.) LAPACK has 24 routines that call `DTRMM` directly. They fall into five categories.

1. Test routines: `DCHK3`, `DCHKE`, `DLARHS`
2. Routines relating to QR factorization or using the result of a QR factorization (especially with block Householder reflectors): `DGELQT3`, `DLARFB`, `DGEQRT3`, `DLARFB_GETT`, `DLARZB`, `DORM22`
3. Routines relating to computing an inverse of a triangular matrix or of a matrix that has been factored into triangular matrices: `DLAUUM`, `DTRITRI`, `DTFTRI`, `DPFTRI`
4. Routines relating to solving eigenvalue (or generalized eigenvalue) problems: `DLAHR2`, `DSYGST`, `DGEHRD`, `DSYGV`, `DSYGV_2STAGE`, `DSYGVD`, `DSYGVX` (note that `DLAQR5` depends on `DTRMM` via `EXTERNAL` declaration, but doesn't actually call it)
5. Routines relating to symmetric indefinite factorizations: `DSYT01_AA`, `DSYTRI2X`, `DSYTRI_3X`

The only routines that call `DTRMM` with `alpha` equal to anything other than one or negative one are the testing routines. Some calls in `DGELQT3` and `DLARFB_GETT` use negative one, but these calls never specify an implicit unit diagonal (they use the explicit diagonal option). The only routine that might possibly call `DTRMM` with both negative one as alpha and the implicit unit diagonal is `DTFTRI`. (This routine "computes the inverse of a triangular matrix A stored in RFP [Rectangular Full Packed] format." RFP format was introduced to LAPACK in the late 2000's, well after the BLAS Standard was published. See LAPACK Working Note 199, which was published in 2008.) `DTFTRI` passes its caller's `diag` argument (which specifies either implicit unit diagonal or explicit diagonal) to `DTRMM`. The only two LAPACK routines that call `DTFTRI` are `DERRRFP` (a testing routine) and `DPFTRI`. `DPFTRI` only ever calls `DTFTRI` with `diag` *not* specifying the implicit unit diagonal option. Therefore, LAPACK never needs both `alpha` not equal to one and the implicit unit

diagonal option, so adding the ability to "scale the implicit diagonal" in [linalg] is a low-priority feature.

4.5.2.6 Fixes would not break backwards compatibility

We can think of two ways to fix this issue. First, we could add an `alpha` scaling parameter, analogous to the symmetric and Hermitian rank-1 and rank-k update functions. Second, we could add a new kind of `Diagonal` template parameter type that expresses a "diagonal value." For example, `implicit_diagonal_t{alpha}` (or a function form, `implicit_diagonal(alpha)`) would tell the algorithm not to access the diagonal elements, but instead to assume that their value is `alpha`. Both of these solutions would let users specify the diagonal's scaling factor separately from the scaling factor for the rest of the matrix. Those two scaling factors could differ, which is new functionality not offered by the BLAS. More importantly, both of these solutions could be added later, after C++26, without breaking backwards compatibility.

4.5.3 Triangular solves with implicit unit diagonals

1. In BLAS, triangular solves with possibly multiple right-hand sides (`xTRSM`) apply `alpha` scaling to the implicit unit diagonal. In [linalg], the scaling factor `alpha` is not applied to the implicit unit diagonal. This is because the library does not interpret `scaled(alpha, A)` differently than any other `mdspan`.

2. Users of triangular solves would need a separate `scale` call to recover BLAS functionality.

3. LAPACK sometimes calls `xTRSM` with `alpha` not equal to one.

4. Straightforward approaches for fixing this issue would not break backwards compatibility.

5. Therefore, we do not consider fixing this a high-priority issue, and we do not propose a fix for it in this paper.

4.5.3.1 BLAS applies alpha after unit diagonal; linalg applies it before

Triangular solves have a similar issue to the one explained in the previous section. The BLAS routine `xTRSM` applies `alpha` "after" the implicit unit diagonal, while std::linalg applies `alpha` "before." (`xTRSV` does not take an `alpha` scaling factor.) As a result, the BLAS solves with a different matrix than std::linalg.

In mathematical terms, `xTRSM` solves the equation $\alpha(A + I)X = B$ for X, where A is the user's input matrix (without implicit unit diagonal) and I is the identity matrix (with ones on the diagonal and zeros everywhere else). `triangular_matrix_matrix_left_solve` solves the equation $(\alpha A + I)Y = B$ for Y. The two results X and Y are not equal in general.

4.5.3.2 Work-around requires changing all elements of the matrix

Users could work around this problem by first scaling the matrix A by α, and then solving for Y. In the common case where the "other triangle" of A holds another triangular matrix, users could not call `scale(alpha, A)`. They would instead need to iterate over the elements of A manually. Users might also need to "unscale" the matrix after the solve. Another option would be to copy the matrix A before scaling.

```
for (size_t i = 0; i < A.extent(0); ++i) {
  for (size_t j = i + 1; j < A.extent(1); ++j) {
    A[i, j] *= alpha;
  }
}
triangular_matrix_matrix_left_solve(A, lower_triangle,
        implicit_unit_diagonal, B, Y);
for (size_t i = 0; i < A.extent(0); ++i) {
  for (size_t j = i + 1; j < A.extent(1); ++j) {
    A[i, j] /= alpha;
  }
}
```

Users cannot solve this problem by scaling B (either with `scaled(1.0 / alpha, B)` or with `scale(1.0 / alpha, B)`). Transforming X into Y or vice versa is mathematically nontrivial in general, and may introduce new failure conditions. This issue occurs with both the in-place and out-of-place triangular solves.

§ 4.5.3.3 Unsupported case occurs in LAPACK

The common case in LAPACK is calling `xTRSM` with `alpha` equal to one, but other values of `alpha` occur. For example, `xTRTRI` calls `xTRSM` with `alpha` equal to −1. Thus, we cannot dismiss this issue, as we could with `xTRMM`.

§ 4.5.3.4 Fixes would not break backwards compatibility

As with triangular matrix products above, we can think of two ways to fix this issue. First, we could add an `alpha` scaling parameter, analogous to the symmetric and Hermitian rank-1 and rank-k update functions. Second, we could add a new kind of `Diagonal` template parameter type that expresses a "diagonal value." For example, `implicit_diagonal_t{alpha}` (or a function form, `implicit_diagonal(alpha)`) would tell the algorithm not to access the diagonal elements, but instead to assume that their value is `alpha`. Both of these solutions would let users specify the diagonal's scaling factor separately from the scaling factor for the rest of the matrix. Those two scaling factors could differ, which is new functionality not offered by the BLAS. More importantly, both of these solutions could be added later, after C++26, without breaking backwards compatibility.

§ 5 Ordering with respect to other proposals and LWG issues

Note that two other `std::linalg` fix papers were voted into the Working Draft for C++26 at the Wrocław, Poland meeting in November 2024.

— P3222R1: "Fix C++26 by adding `transposed` special cases for P2642 layouts"

— P3050R3: "Fix C++26 by optimizing `linalg::conjugated` for noncomplex value types"

LEWG was aware of these two papers and this proposal P3371 in its 2024-09-03 review of P3050R2. All three of these papers increment the value of the `__cpp_lib_linalg` macro. While this technically causes a conflict between the papers, advice from LEWG on 2024-09-03 was not to introduce special wording changes to avoid this conflict.

Two outstanding LWG issues for std::linalg remain.

— LWG4136 specifies the behavior of Hermitian algorithms on diagonal matrix elements with nonzero imaginary part. LWG4136's proposed fix adds a sentence to [linalg.general]. P3371 does not change [linalg.general]. However, P3371 does add updating overloads of Hermitian algorithms that take another Hermitian matrix parameter `E`. The proposed fix in LWG4136 does not cover parameters named `E`, so we apply the analog of that fix to the wording here in P3371.

— LWG4137, "Fix Mandates, Preconditions, and Complexity elements of [linalg] algorithms," affects [linalg.algs.blas3.rankk] and [linalg.algs.blas3.rank2k], which this proposal also changes. We rebase this proposal's changes atop the wording changes in LWG4137's proposed fix. It is our view that the two sets of changes do not conflict in a mathematical or design specification sense. Note that this proposal only applies the proposed fixes in LWG4137 to the relevant clauses [linalg.algs.blas3.rankk] and [linalg.algs.blas3.rank2k]. Thus, P3371 does not (completely) resolve LWG4137.

§ 6 Implementation status

Pull Request 293 in the reference std::linalg implementation implements the proposed changes. It also adds tests to ensure test coverage of the new overloads.

§ 7 Acknowledgments

Many thanks (with permission) to Raffaele Solcà (CSCS Swiss National Supercomputing Centre, `raffaele.solca@cscs.ch`) for pointing out some of the issues fixed by this paper, as well as the issues leading to LWG4137.

§ 8 Wording

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The 🔲 character is used to denote a

placeholder section number which the editor shall determine. Additions are shown in green, and removals in red.

In **[version.syn]**, for the following definition,

```
#define __cpp_lib_linalg YYYYMML // also in <linalg>
```

adjust the placeholder value `YYYYMML` as needed so as to denote this proposal's date of adoption.

§ 8.1 Constrain all `class Scalar` template parameters

[Editor's note: Throughout [linalg], if a function declaration, class declaration, function definition, or class definition has a `Scalar` template parameter, replace `class Scalar` with *`scalar`* `Scalar`. Please apply this change *after* the changes below. That is, in the changes below as well as in the current Working Draft, replace `class Scalar` with *`scalar`* `Scalar`.]

[Editor's note: We express the change this way in order to minimize the diff to review. The exposition-only concept *`scalar`* will be declared and defined in the rest of the diff below.]

§ 8.2 Change exposition-only concepts

Change the header `<linalg>` synopsis **[linalg.syn]** as follows.

Just after the declaration of the exposition-only concept *`inout-matrix`*, replace the exposition-only concept *`possibly-packed-inout-matrix`* with the new exposition-only concept *`possibly-packed-out-matrix`*.

At the end of the list of declarations of exposition-only concepts, after the declaration of the exposition-only concept *`inout-object`*, add a declaration of the new exposition-only concept *`scalar`*.

```
  template<class T>
    concept inout-matrix = see below;                    // exposition only
  template<class T>
    concept possibly-packed-inout-matrix = see below;    // exposition only
  template<class T>
    concept in-object = see below;                       // exposition only
  template<class T>
    concept out-object = see below;                      // exposition only
  template<class T>
    concept inout-object = see below;                    // exposition only
```

```
  template<class T>
    concept scalar = see below;                          // exposition only
```

Then, in **[linalg.helpers.concepts]** paragraph 1, just after the definition of the exposition-only variable template *`is-layout-blas-packed`*, replace the

exposition-only concept *possibly-packed-inout-matrix* with the new exposition-only concept *possibly-packed-out-matrix*. The two concepts have the same definitions and thus differ only in name.

```
  template<class T>
    constexpr bool is-layout-blas-packed = false; // exposition only

  template<class Triangle, class StorageOrder>
    constexpr bool is-layout-blas-packed<layout_blas_packed<Triangle,
        StorageOrder>> = true;

  template<class T>
    concept possibly-packed-inout-matrix =
      is-mdspan<T> && T::rank() == 2 &&
      is_assignable_v<typename T::reference, typename T::element_type> &&
      (T::is_always_unique() || is-layout-blas-packed<typename
        T::layout_type>);
```

Then, again in **[linalg.helpers.concepts]** paragraph 1, after the definition of the exposition-only concept *inout-object*, add the definition of the new exposition-only concept *scalar* as specified below.

```
template<class T>
  concept inout-object =
    is-mdspan<T> && (T::rank() == 1 || T::rank() == 2);
```

```
template<class T>
  concept scalar =
    semiregular<T> && (! is-mdspan<T>) && (! is_execution_policy_v<T>);
```

Then, in [linalg.helpers.concepts], change paragraph 3 to rename *possibly-packed-inout-matrix* to *possibly-packed-out-matrix*.

Unless explicitly permitted, any *inout-vector*, *inout-matrix*, *inout-object*, *out-vector*, *out-matrix*, *out-object*, or *possibly-packed-inout-matrix* parameter of a function in [linalg] shall not overlap any other `mdspan` parameter of the function.

§ 8.3 Rank-1 update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, change the declarations of the `matrix_rank_1_update`, `matrix_rank_1_update_c`, `symmetric_matrix_rank_1_update`, and `hermitian_matrix_rank_1_update` overloads as follows. *[Editorial Note:* There are three sets of changes here.

1. The existing overloads become "overwriting" overloads.
2. New "updating" overloads are added.
3. Any overloads of the symmetric and Hermitian rank-1 update functions that do *not* have a `Scalar alpha` parameter are removed. ("General" rank-1 update functions `matrix_rank_1_update` and

`matrix_rank_1_update_c` have never had overloads that take `Scalar alpha`.) – *end note]*

```
// [linalg.algs.blas2.rank1], nonsymmetric rank-1 matrix update

// overwriting nonsymmetric rank-1 matrix update
template<in-vector InVec1, in-vector InVec2, inout-matrix InOutMat>
  void matrix_rank_1_update(InVec1 x, InVec2 y, InOutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
      inout-matrix InOutMat>
  void matrix_rank_1_update(ExecutionPolicy&& exec,
                            InVec1 x, InVec2 y, InOutMat A);

template<in-vector InVec1, in-vector InVec2, inout-matrix InOutMat>
  void matrix_rank_1_update_c(InVec1 x, InVec2 y, InOutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
      inout-matrix InOutMat>
  void matrix_rank_1_update_c(ExecutionPolicy&& exec,
                              InVec1 x, InVec2 y, InOutMat A);
```

```
// updating nonsymmetric rank-1 matrix update
template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix
      OutMat>
  void matrix_rank_1_update(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-matrix InMat, in-
      vector InVec2, out-matrix OutMat>
  void matrix_rank_1_update(ExecutionPolicy&& exec,
                            InVec1 x, InVec2 y, InMat E, OutMat A);

template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix
      OutMat>
  void matrix_rank_1_update_c(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-
      matrix InMat, out-matrix OutMat>
  void matrix_rank_1_update_c(ExecutionPolicy&& exec,
                              InVec1 x, InVec2 y, InMat E, OutMat A);
```

```
// [linalg.algs.blas2.symherrank1], symmetric or Hermitian rank-1 matrix
      update

// overwriting symmetric rank-1 matrix update
template<class Scalar, in-vector InVec, possibly-packed-inout-matrix
      InOutMat, class Triangle>
  void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, InOutMat A,
      Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, possibly-packed-inout-matrix
      InOutMat, class Triangle>
  void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, InOutMat A,
      Triangle t);
```

```
template<in-vector InVec, possibly-packed-inout-matrix InOutMat, class
      Triangle>
  void symmetric_matrix_rank_1_update(InVec x, InOutMat A, Triangle t);
template<class ExecutionPolicy,
```

```
          in-vector InVec, possibly-packed-inout-matrix InOutMat, class
      Triangle>
  void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      InVec x, InOutMat A, Triangle t);
```

```
// overwriting Hermitian rank-1 matrix update
template<class Scalar, in-vector InVec, possibly-packed-~~in~~out-matrix
      ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, ~~In~~OutMat A,
      Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, possibly-packed-~~in~~out-matrix
      ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, ~~In~~OutMat A,
      Triangle t);
```

```
template<in-vector InVec, possibly-packed-inout-matrix InOutMat, class
      Triangle>
  void hermitian_matrix_rank_1_update(InVec x, InOutMat A, Triangle t);
template<class ExecutionPolicy,
         in-vector InVec, possibly-packed-inout-matrix InOutMat, class
      Triangle>
  void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      InVec x, InOutMat A, Triangle t);
```

```
// updating symmetric rank-1 matrix update
template<class Scalar, in-vector InVec, in-matrix InMat, possibly-
      packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, InMat E,
      OutMat A, Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, in-matrix InMat, possibly-
      packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, InMat E,
      OutMat A, Triangle t);

// updating Hermitian rank-1 matrix update
template<class Scalar, in-vector InVec, in-matrix InMat, possibly-
      packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, InMat E,
      OutMat A, Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, in-matrix InMat, possibly-
      packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, InMat E,
      OutMat A, Triangle t);
```

§ 8.4 Rank-2 update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, change the declarations of the `symmetric_matrix_rank_2_update` and `hermitian_matrix_rank_2_update`

overloads as follows. *[Editorial Note:* There are two sets of changes here.

1. The existing overloads become "overwriting" overloads.

2. New "updating" overloads are added. – *end note]*

```
// [linalg.algs.blas2.rank2], symmetric and Hermitian rank-2 matrix
      updates

// overwriting symmetric rank-2 matrix update
template<in-vector InVec1, in-vector InVec2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat, class Triangle>
  void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, ~~In~~OutMat A,
      Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat, class Triangle>
  void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, ~~In~~OutMat A,
      Triangle t);

// overwriting Hermitian rank-2 matrix update
template<in-vector InVec1, in-vector InVec2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, ~~In~~OutMat A,
      Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, ~~In~~OutMat A,
      Triangle t);
```

```
// updating symmetric rank-2 matrix update
template<in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E,
      OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, InMat E,
      OutMat A, Triangle t);

// updating Hermitian rank-2 matrix update
template<in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E,
      OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, InMat E,
      OutMat A, Triangle t);
```

§ 8.5 Rank-k update functions in synopsis

In the header <linalg> synopsis **[linalg.syn]**, update the declarations of the `symmetric_matrix_rank_k_update` and `hermitian_matrix_rank_k_update` overloads as follows. *[Editorial Note:* There are three sets of changes here.

1. The existing overloads become "overwriting" overloads.
2. New "updating" overloads are added.
3. Any overloads of the symmetric and Hermitian rank-k update functions that do *not* have a `Scalar alpha` parameter are removed. – *end note]*

```
// [linalg.algs.blas3.rankk], rank-k update of a symmetric or Hermitian
      matrix

// overwriting rank-k symmetric matrix update
template<class Scalar,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void symmetric_matrix_rank_k_update(Scalar alpha, InMat A, InOutMat C,
      Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InMat A, InOutMat C,
      Triangle t);
```

```
template<in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void symmetric_matrix_rank_k_update(InMat A, InOutMat C, Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec,
                                      InMat A, InOutMat C, Triangle t);
```

```
// overwriting rank-k Hermitian matrix update
template<class Scalar,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void hermitian_matrix_rank_k_update(Scalar alpha, InMat A, InOutMat C,
      Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
```

```
  void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InMat A, InOutMat C,
      Triangle t);
```

```
template<in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void hermitian_matrix_rank_k_update(InMat A, InOutMat C, Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
  void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec,
                                      InMat A, InOutMat C, Triangle t);
```

```
// updating rank-k symmetric matrix update
template<class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
  void symmetric_matrix_rank_k_update(
    Scalar alpha,
    InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
  void symmetric_matrix_rank_k_update(
    ExecutionPolicy&& exec, Scalar alpha,
    InMat1 A, InMat2 E, OutMat C, Triangle t);

// updating rank-k Hermitian matrix update
template<class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
  void hermitian_matrix_rank_k_update(
    Scalar alpha,
    InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
  void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec, Scalar alpha,
    InMat1 A, InMat2 E, OutMat C, Triangle t);
```

§ 8.6 Rank-2k update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, update the declarations of the `symmetric_matrix_rank_2k_update` and

`hermitian_matrix_rank_2k_update` overloads as follows. *[Editorial Note:* There are two sets of changes here.

1. The existing overloads become "overwriting" overloads.

2. New "updating" overloads are added. – *end note]*

```
// [linalg.algs.blas3.rank2k], rank-2k update of a symmetric or
      Hermitian matrix

// overwriting rank-2k symmetric matrix update
template<in-matrix InMat1, in-matrix InMat2,
         possibly-packed-inout-matrix InOutMat, class Triangle>
  void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B, InOutMat C,
      Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1, in-matrix InMat2,
         possibly-packed-inout-matrix InOutMat, class Triangle>
  void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                       InMat1 A, InMat2 B, InOutMat C,
      Triangle t);

// overwriting rank-2k Hermitian matrix update
template<in-matrix InMat1, in-matrix InMat2,
         possibly-packed-inout-matrix InOutMat, class Triangle>
  void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B, InOutMat C,
      Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1, in-matrix InMat2,
         possibly-packed-inout-matrix InOutMat, class Triangle>
  void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                       InMat1 A, InMat2 B, InOutMat C,
      Triangle t);
```

```
// updating symmetric rank-2k matrix update
template<in-matrix InMat1, in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B, InMat3 E,
      OutMat C, Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1, in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                       InMat1 A, InMat2 B, InMat3 E,
      OutMat C, Triangle t);

// updating Hermitian rank-2k matrix update
template<in-matrix InMat1, in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B, InMat3 E,
      OutMat C, Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1, in-matrix InMat2,
```

```
          in-matrix InMat3,
          possibly-packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                       InMat1 A, InMat2 B, InMat3 E,
      OutMat C, Triangle t);
```

§ 8.7 Constrain linear algebra value type

Change [linalg.reqs.val] as follows.

1 Throughout [linalg], the following types are *linear algebra value types*:

(1.1) — the `value_type` type alias of any input or output `mdspan` parameter(s) of any function in **[linalg]**; and

(1.2) — the `Scalar` template parameter (if any) of any function or class in **[linalg]**.

2 Linear algebra value types shall model ~~`semiregular`~~*`scalar`*.

3 A value-initialized object of linear algebra value type shall act as the additive identity.

§ 8.8 Specification of nonsymmetric rank-1 update functions

Change [linalg.algs.blas2.rank1] as follows.

1 The following elements apply to all functions in [linalg.algs.blas2.rank1].

2 *Mandates*:

(2.1) *`possibly-multipliable`*`<OutMat, InVec2, InVec1>()` is `true`, and

(2.2) *`possibly-addable`*`<OutMat, InMat, OutMat>()` is `true` for those overloads with an `E` parameter.

3 *Preconditions*:

(3.1) *`multipliable`*`(A, y, x)` is `true`, and

(3.2) *`addable`*`(A, E, A)` is `true` for those overloads with an `E` parameter.

4 *Complexity*: *O*(`x.extent(0)` × `y.extent(0)`).

```
template<in-vector InVec1, in-vector InVec2, ~~in~~out-matrix ~~In~~OutMat>
  void matrix_rank_1_update(InVec1 x, InVec2 y, ~~In~~OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, ~~in~~out-
        matrix ~~In~~OutMat>
  void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x, InVec2 y,
        ~~In~~OutMat A);
```

5 These functions perform an overwriting nonsymmetric nonconjugated rank-1 update.

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

[Note: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERU` (for complex element types). – *end note]*

2 *Mandates*: `possibly_multipliable<InOutMat, InVec2, InVec1>()` is `true`.

3 *Preconditions*: `multipliable(A, y, x)` is `true`.

4 *Effects*: Computes a matrix A' such that $A' = A + xy^T$, and assigns each element of A' to the corresponding element of A.

6 *Effects*: Computes $A = xy^T$.

5 *Complexity*: $O($ `x.extent(0)` $\times$ `y.extent(0)` $)$.

```
template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix
        OutMat>
  void matrix_rank_1_update(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-
        matrix InMat, out-matrix OutMat>
  void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x, InVec2 y,
        InMat E, OutMat A);
```

7 These functions perform an updating nonsymmetric nonconjugated rank-1 update.

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

[Note: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERU` (for complex element types). – *end note]*

8 *Effects*: Computes $A = E + xy^T$.

9 *Remarks*: `A` may alias `E`.

```
template<in-vector InVec1, in-vector InVec2, ~~in~~out-matrix ~~In~~OutMat>
  void matrix_rank_1_update_c(InVec1 x, InVec2 y, ~~In~~OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, ~~in~~out-
        matrix ~~In~~OutMat>
  void matrix_rank_1_update_c(ExecutionPolicy&& exec, InVec1 x, InVec2 y,
        ~~In~~OutMat A);
```

10 These functions perform an overwriting nonsymmetric conjugated rank-1 update.

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

[Note: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERC` (for complex element types). – *end note]*

11 *Effects*:

(11.1) For the overloads without an `ExecutionPolicy` argument, equivalent to:

```
matrix_rank_1_update(x, conjugated(y), A);
```

(11.2) otherwise, equivalent to:

```
matrix_rank_1_update(std::forward<ExecutionPolicy>(exec), x,
        conjugated(y), A);
```

```
template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix
        OutMat>
  void matrix_rank_1_update_c(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-
        matrix InMat, out-matrix OutMat>
  void matrix_rank_1_update_c(ExecutionPolicy&& exec, InVec1 x, InVec2 y,
        InMat E, OutMat A);
```

12 These functions perform an updating nonsymmetric conjugated rank-1 update.

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

[Note: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERC` (for complex element types). – *end note]*

13 *Effects*:

(13.1) For the overloads without an `ExecutionPolicy` argument, equivalent to:

```
matrix_rank_1_update(x, conjugated(y), E, A);
```

(13.2) otherwise, equivalent to:

```
matrix_rank_1_update(std::forward<ExecutionPolicy>(exec), x,
        conjugated(y), E, A);
```

§ 8.9 Specification of symmetric and Hermitian rank-1 update functions

Change [linalg.algs.blas2.symherrank1] as follows.

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

1 *[Note:* These functions correspond to the BLAS functions `xSYR`, `xSPR`, `xHER`, and `xHPR`. They take a scaling factor `alpha`, because it would be impossible to express the update $A = A - xx^T$ in noncomplex arithmetic otherwise. – *end note]*

2 The following elements apply to all functions in [linalg.algs.blas2.symherrank1].

3 For any function `F` in this subclause with a parameter named `t`, an `InMat` template parameter, and a function parameter `InMat E`, `t` applies to accesses done through the parameter `E`. `F` only accesses the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` only uses the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` only uses the value

(3.1) — *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(3.2) — `E[j, i]` if the name of `F` starts with `symmetric`.

4 *Mandates*:

(4.1) — If ~~`In`~~`OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.2) — If the function has an `InMat` template parameter and `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.3) — *`compatible-static-extents`*`<decltype(A), decltype(A)>(0, 1)` is `true`; ~~and~~

(4.4) — *`compatible-static-extents`*`<decltype(A), decltype(x)>(0, 0)` is `true`~~.~~; and

(4.5) — *`possibly-addable`*`<decltype(A), decltype(E), decltype(A)>()` is `true` for those overloads with an `E` parameter.

5 *Preconditions*:

(5.1) — `A.extent(0)` equals `A.extent(1)`, ~~and~~

(5.2) — `A.extent(0)` equals `x.extent(0)`~~.~~, and

(5.3) — *`addable`*`(A, E, A)` is `true` for those overloads with an `E` parameter.

6 *Complexity*: *O*(`x.extent(0)` × `x.extent(0)`).

```
template<class Scalar, in-vector InVec, possibly-packed-~~in~~out-matrix
         ~~In~~OutMat, class Triangle>
  void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, ~~In~~OutMat A,
         Triangle t);
template<class ExecutionPolicy,
          class Scalar, in-vector InVec, possibly-packed-~~in~~out-matrix
         ~~In~~OutMat, class Triangle>
  void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, ~~In~~OutMat A,
         Triangle t);
```

7 These functions perform an overwriting symmetric rank-1 update of the symmetric matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

8 *Effects*: Computes ~~a matrix A' such that $A' = A + \alpha xx^T$~~$A = \alpha xx^T$, where the scalar α is `alpha`~~, and assigns each element of A' to the corresponding element of A~~.

```
template<in-vector InVec, possibly-packed-inout-matrix InOutMat, class
        Triangle>
  void symmetric_matrix_rank_1_update(InVec x, InOutMat A, Triangle t);
template<class ExecutionPolicy,
         in-vector InVec, possibly-packed-inout-matrix InOutMat, class
        Triangle>
  void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      InVec x, InOutMat A, Triangle t);
```

9 These functions perform a symmetric rank-1 update of the symmetric matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

10 *Effects*: Computes a matrix A' such that $A' = A + xx^T$ and assigns each element of A' to the corresponding element of A.

```
template<class Scalar, in-vector InVec, in-matrix InMat, possibly-packed-
        out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, InMat E,
        OutMat A, Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, in-matrix InMat, possibly-packed-
        out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, InMat E,
        OutMat A, Triangle t);
```

9 These functions perform an updating symmetric rank-1 update of the symmetric matrix `A` using the symmetric matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

10 *Effects*: Computes $A = E + \alpha xx^T$, where the scalar α is `alpha`.

11 *Remarks*: `A` may alias `E`.

```
template<class Scalar, in-vector InVec, possibly-packed-~~in~~out-matrix
        ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, ~~In~~OutMat A,
        Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, possibly-packed-~~in~~out-matrix
        ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, ~~In~~OutMat A,
        Triangle t);
```

12 These functions perform an overwriting Hermitian rank-1 update of the Hermitian matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

13 *Effects*: Computes ~~a matrix A' such that $A' = A + \alpha xx^H$~~$A = \alpha xx^H$, where the scalar α is ~~`alpha`~~*real-if-needed*(alpha)~~, and assigns each element of A' to the corresponding~~

~~element of *A*~~.

```
template<in-vector InVec, possibly-packed-inout-matrix InOutMat, class
        Triangle>
  void hermitian_matrix_rank_1_update(InVec x, InOutMat A, Triangle t);
template<class ExecutionPolicy,
         in-vector InVec, possibly-packed-inout-matrix InOutMat, class
        Triangle>
  void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      InVec x, InOutMat A, Triangle t);
```

13 These functions perform a Hermitian rank-1 update of the Hermitian matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

14 *Effects*: Computes a matrix A' such that $A' = A + xx^H$ and assigns each element of A' to the corresponding element of A.

```
template<class Scalar, in-vector InVec, in-matrix InMat, possibly-packed-
        out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, InMat E,
        OutMat A, Triangle t);
template<class ExecutionPolicy,
         class Scalar, in-vector InVec, in-matrix InMat, possibly-packed-
        out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                      Scalar alpha, InVec x, InMat E,
        OutMat A, Triangle t);
```

14 These functions perform an updating Hermitian rank-1 update of the Hermitian matrix `A` using the Hermitian matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

15 *Effects*: Computes $A = E + \alpha xx^H$, where the scalar α is *real-if-needed*`(alpha)`.

16 *Remarks*: `A` may alias `E`.

§ 8.10 Specification of symmetric and Hermitian rank-2 update functions

Change [linalg.algs.blas2.rank2] as follows.

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

1 *[Note:* These functions correspond to the BLAS functions `xSYR2`, `xSPR2`, `xHER2`, and `xHPR2`. – *end note]*

2 The following elements apply to all functions in [linalg.algs.blas2.rank2].

3 For any function `F` in this subclause with a parameter named `t`, an `InMat` template parameter, and a function parameter `InMat E`, `t` applies to accesses done through the

parameter `E`. `F` only accesses the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` only uses the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` only uses the value

(3.1) — *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(3.2) — `E[j, i]` if the name of `F` starts with `symmetric`.

4 *Mandates*:

(4.1) — If ~~`In`~~`OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.2) — If the function has an `InMat` template parameter and `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.3) — *`compatible-static-extents`*`<decltype(A), decltype(A)>(0, 1)` is `true`;

(4.4) — *`possibly-multipliable`*`<decltype(A), decltype(x), decltype(y)>()` is `true`~~.~~;
and

(4.5) — *`possibly-addable`*`<decltype(A), decltype(E), decltype(A)>()` is `true` for those overloads with an `E` parameter.

5 *Preconditions*:

(5.1) — `A.extent(0)` equals `A.extent(1)`, ~~and~~

(5.2) — *`multipliable`*`(A, x, y)` is `true`~~.~~, and

(5.3) — *`addable`*`(A, E, A)` is `true` for those overloads with an `E` parameter.

6 *Complexity*: *O*(`x.extent(0)` × `y.extent(0)`).

```
template<in-vector InVec1, in-vector InVec2,
         possibly-packed-inout-matrix InOutMat, class Triangle>
  void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InOutMat A,
         Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         possibly-packed-inout-matrix InOutMat, class Triangle>
  void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, InOutMat A,
         Triangle t);
```

7 These functions perform a~~n~~ overwriting symmetric rank-2 update of the symmetric matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

8 *Effects*: Computes ~~A' such that $A' = A + xy^T + yx^T$ and assigns each element of A' to the corresponding element of A~~$A = xy^T + yx^T$.

```
template<in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E, OutMat
         A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, InMat E, OutMat
         A, Triangle t);
```

9 These functions perform an updating symmetric rank-2 update of the symmetric matrix `A` using the symmetric matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

10 *Effects*: Computes $A = E + xy^T + yx^T$.

11 *Remarks*: `A` may alias `E`.

```
template<in-vector InVec1, in-vector InVec2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, ~~In~~OutMat A,
         Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, ~~In~~OutMat A,
         Triangle t);
```

12 These functions perform an overwriting Hermitian rank-2 update of the Hermitian matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

13 *Effects*: Computes ~~A' such that $A' = A + xy^H + yx^H$ and assigns each element of A' to the corresponding element of A~~$A = xy^H + yx^H$.

```
template<in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E, OutMat
         A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat, class Triangle>
  void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, InMat E, OutMat
         A, Triangle t);
```

14 These functions perform an updating Hermitian rank-2 update of the Hermitian matrix `A` using the Hermitian matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

15 *Effects*: Computes $A = E + xy^H + yx^H$.

16 *Remarks*: `A` may alias `E`.

§ 8.11 Specification of rank-k update functions

Change [linalg.algs.blas3.rankk] as follows.

[Editorial Note: The changes proposed here are rebased atop the changes proposed in LWG4137, "Fix Mandates, Preconditions, and Complexity elements of [linalg] algorithms." – *end note]*

[Note: These functions correspond to the BLAS functions `xSYRK` and `xHERK`. – *end note]*

1 The following elements apply to all functions in [linalg.algs.blas3.rankk].

2 For any function `F` in this subclause with a parameter named `t`, an `InMat2` template parameter, and a function parameter `InMat2 E`, `t` applies to accesses done through the parameter `E`. `F` only accesses the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` only uses the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` only uses the value

(2.1) — *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(2.2) — `E[j, i]` if the name of `F` starts with `symmetric`.

3 *Mandates:*

(3.1) — If ~~`In`~~`OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(3.2) — If the function has an `InMat2` template parameter and if `InMat2` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument.

(3.3) — *`possibly-multipliable`*`<decltype(A), decltype(transposed(A)), decltype(C)>()` ~~*`compatible-static-extents`*`<decltype(A), decltype(A)>(0, 1)`~~ is `true`; and

(3.3) — *`compatible-static-extents`*`<decltype(C), decltype(C)>(0, 1)` is `true`; and

(3.4) — *`compatible-static-extents`*`<decltype(A), decltype(C)>(0, 0)` is `true`.

(3.4) — *`possibly-addable`*`<decltype(C), decltype(E), decltype(C)>()` is `true` for those overloads with an `E` parameter.

4 *Preconditions:*

(4.1) — `A.extent(0)` equals `A.extent(1)`,

(4.2) — `C.extent(0)` equals `C.extent(1)`, and

(4.3) — `A.extent(0)` equals `C.extent(0)`.

(4.1) — *`multipliable`*`(A, transposed(A), C)` is `true`; and *[Note:* This implies that `C` is square – *end note]*

(4.2) — *`addable`*`(C, E, C)` is `true` for those overloads with an `E` parameter.

5 *Complexity:* *O*(`A.extent(0)` · `A.extent(1)` · `AC.extent(0)`).

6 *Remarks:* `C` may alias `E` for those overloads with an `E` parameter.

```
template<class Scalar,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  Scalar alpha,
  InMat A,
  InOutMat C,
  Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  Scalar alpha,
  InMat A,
  InOutMat C,
  Triangle t);
```

7 *Effects:* Computes ~~a matrix C' such that $C' = C + \alpha AA^T$~~ $C = \alpha AA^T$, where the scalar α is `alpha`~~, and assigns each element of C' to the corresponding element of C.~~.

```
template<in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  InMat A,
  InOutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  InMat A,
  InOutMat C,
  Triangle t);
```

7 *Effects:* Computes a matrix C' such that $C' = C + AA^T$, and assigns each element of C' to the corresponding element of C.

```
template<class Scalar,
         in-matrix InMat,
         possibly-packed-~~in~~out-matrix ~~In~~InOutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  Scalar alpha,
  InMat A,
  ~~In~~OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  Scalar alpha,
  InMat A,
  ~~In~~OutMat C,
  Triangle t);
```

8 *Effects:* Computes ~~a matrix C' such that $C' = C + \alpha AA^H$~~$C = \alpha AA^H$, where the scalar α is ~~alpha~~*real-if-needed*(alpha)~~, and assigns each element of C' to the corresponding element of C~~..

```
template<in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  InMat A,
  InOutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-inout-matrix InOutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  InMat A,
  InOutMat C,
  Triangle t);
```

9 *Effects:* Computes a matrix C' such that $C' = C + AA^H$, and assigns each element of C' to the corresponding element of C.

```
template<class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
```

```
void symmetric_matrix_rank_k_update(
  Scalar alpha,
  InMat1 A,
  InMat2 E,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  Scalar alpha,
  InMat1 A,
  InMat2 E,
  OutMat C,
  Triangle t);
```

9 *Effects:* Computes $C = E + \alpha AA^T$, where the scalar α is `alpha`.

```
template<class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  Scalar alpha,
  InMat1 A,
  InMat2 E,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  Scalar alpha,
  InMat1 A,
  InMat2 E,
  OutMat C,
  Triangle t);
```

10 *Effects:* Computes $C = E + \alpha AA^H$, where the scalar α is *`real-if-needed`*`(alpha)`.

§ 8.12 Specification of rank-2k update functions

Change [linalg.algs.blas3.rank2k] as follows.

[Editorial Note: The changes proposed here are rebased atop the changes proposed in LWG4137, "Fix Mandates, Preconditions, and Complexity elements of [linalg] algorithms." – *end note]*

[Editor's note: The Note below has a bibliography link, which we don't know how to render here and have therefore omitted.]

1 *[Note:* These functions correspond to the BLAS functions `xSYR2K` and `xHER2K`. – *end note]*

2 The following elements apply to all functions in [linalg.algs.blas3.rank2k].

3 For any function `F` in this subclause with a parameter named `t`, an `InMat3` template parameter, and a function parameter `InMat3 E`, `t` applies to accesses done through the parameter `E`. `F` only accesses the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` only uses the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` only uses the value

(3.1) — *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(3.2) — `E[j, i]` if the name of `F` starts with `symmetric`.

4 *Mandates:*

(4.1) — If ~~`In`~~`OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.2) — If the function has an `InMat3` template parameter and if `InMat3` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.2) — *`possibly-multipliable`*`<decltype(A), decltype(transposed(B)), decltype(C)>()` ~~*`possibly-addable`*`<decltype(A), decltype(B), decltype(C)>()`~~ is `true`; ~~and~~

(4.3) — *`possibly-multipliable`*`<decltype(B), decltype(transposed(A)), decltype(C)>()`~~*`compatible-static-extents`*`<decltype(A), decltype(A)>(0, 1)`~~ is `true`~~.~~; and

(4.5) — *`possibly-addable`*`<decltype(C), decltype(E), decltype(C)>()` is `true` for those overloads with an `E` parameter.

[Editorial Note: The proposed fix for LWG4137, "Fix Mandates, Preconditions, and Complexity elements of [linalg] algorithms," incorrectly adds `(0, 1)` after *`possibly-multipliable`*`<decltype(B), decltype(transposed(A)), decltype(C)>()` in paragraph 4.3 above (3.3 in the issue). – *end note]*

5 *Preconditions:*

(5.1) — *multipliable*(A, transposed(B), C) ~~*addable(A, B, C)*~~ is `true`, and

(5.2) — *multipliable*(B, transposed(A), C) is `true` ~~A.extent(0) equals A.extent(1)~~. *[Note:* This and the previous imply that `C` is square. *– end note]*

(5.3) — *addable*(C, E, C) is `true` for those overloads with an `E` parameter.

6 *Complexity: O*(`A.extent(0)` · `A.extent(1)` · `B`~~`C`~~`.extent(0)`)

7 *Remarks:* `C` may alias `E` for those overloads with an `E` parameter.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  ~~In~~OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
  ~~In~~OutMat C,
  Triangle t);
```

8 *Effects:* Computes ~~a matrix C' such that $C' = C + AB^T + BA^T$, and assigns each element of C' to the corresponding element of C~~$C = AB^T + BA^T$.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  ~~In~~OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-~~in~~out-matrix ~~In~~OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
```

```
  InOutMat C,
  Triangle t);
```

9 *Effects:* Computes a matrix C' such that $C' = C + AB^H + BA^H$, and assigns each element of C' to the corresponding element of $C$$C = AB^H + BA^H$.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
```

10 *Effects:* Computes $C = E + AB^T + BA^T$.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
```

```
  InMat3 E,
  OutMat C,
  Triangle t);
```

11 *Effects:* Computes $C = E + AB^H + BA^H$.

§ 9 Appendix A: Example of a generic numerical algorithm

The following example shows how to implement a generic numerical algorithm, `two_norm_abs`. This algorithm computes the absolute value of a variety of number types. For complex numbers, it returns the magnitude, which is the same as the two-norm of the two-element vector composed of the real and imaginary parts of the complex number. This Compiler Explorer link demonstrates the implementation. This is not meant to show an ideal implementation. (A better one would use rescaling, in the manner of `std::hypot`, to avoid undue overflow or underflow.) Instead, it illustrates generic numerical algorithm development. Commenting out the `#define DEFINE_CONJ_REAL_IMAG_FOR_REAL 1` line shows that without ADL-findable `conj`, `real`, and `imag`, users' generic numerical algorithms would need more special cases and more assumptions on number types.

```
#include <cassert>
#include <cmath>
#include <complex>
#include <concepts>
#include <type_traits>

#define DEFINE_CONJ_REAL_IMAG_FOR_REAL 1

template<class T>
constexpr bool is_std_complex = false;
template<class R>
constexpr bool is_std_complex<std::complex<R>> = true;

template<class T>
auto two_norm_abs(T t) {
  if constexpr (std::is_unsigned_v<T>) {
    return t;
  }
  else if constexpr (std::is_arithmetic_v<T> || is_std_complex<T>) {
    return std::abs(t);
  }
#if ! defined(DEFINE_CONJ_REAL_IMAG_FOR_REAL)
  else if constexpr (requires(T x) {
      {abs(x)} -> std::convertible_to<T>;
    }) {
    return T{abs(t)};
  }
#endif
  else if constexpr (requires(T x) {
      {sqrt(real(x * conj(x)))} -> std::same_as<decltype(real(x))>;
    }) {
```

```
    return sqrt(real(t * conj(t)));
  }
  else {
    static_assert(false, "No reasonable way to implement abs(t)");
  }
}

struct MyRealNumber {
  MyRealNumber() = default;
  MyRealNumber(double value) : value_(value) {}

  double value() const {
    return value_;
  }

  friend bool operator==(MyRealNumber, MyRealNumber) = default;
  friend MyRealNumber operator-(MyRealNumber x) {
    return {-x.value_};
  }
  friend MyRealNumber operator+(MyRealNumber x, MyRealNumber y) {
    return {x.value_ + y.value_};
  }
  friend MyRealNumber operator-(MyRealNumber x, MyRealNumber y) {
    return {x.value_ - y.value_};
  }
  friend MyRealNumber operator*(MyRealNumber x, MyRealNumber y) {
    return x.value_ * y.value_;
  }

#if defined(DEFINE_CONJ_REAL_IMAG_FOR_REAL)
  friend MyRealNumber conj(MyRealNumber x) { return x; }
  friend MyRealNumber real(MyRealNumber x) { return x; }
  friend MyRealNumber imag(MyRealNumber x) { return {}; }
#else
  friend MyRealNumber abs(MyRealNumber x) { return std::abs(x.value_); }
#endif
  friend MyRealNumber sqrt(MyRealNumber x) { return std::sqrt(x.value_); }

private:
  double value_{};
};

class MyComplexNumber {
public:
  MyComplexNumber(MyRealNumber re, MyRealNumber im = {}) : re_(re),
        im_(im) {}
  MyComplexNumber() = default;

  std::complex<double> value() const {
    return {re_.value(), im_.value()};
  }

  friend bool operator==(MyComplexNumber, MyComplexNumber) = default;
  friend MyComplexNumber operator*(MyComplexNumber z, MyComplexNumber w) {
    return {real(z) * real(w) - imag(z) * imag(w),
      real(z) * imag(w) + imag(z) * real(w)};
```

```
  }
  friend MyComplexNumber conj(MyComplexNumber z) { return {real(z), -
        imag(z)}; }
  friend MyRealNumber real(MyComplexNumber z) { return z.re_; }
  friend MyRealNumber imag(MyComplexNumber z) { return z.im_; }

private:
  MyRealNumber re_{};
  MyRealNumber im_{};
};

int main() {
  [[maybe_unused]] double x1 = two_norm_abs(-4.2);
  assert(x1 == 4.2);

  [[maybe_unused]] float y0 = two_norm_abs(std::complex<float>{-3.0f,
        4.0f});
  assert(y0 == 5.0f);

  [[maybe_unused]] MyRealNumber r = two_norm_abs(MyRealNumber{-6.7});
  assert(r == MyRealNumber{6.7});

  [[maybe_unused]] MyRealNumber z = two_norm_abs(MyComplexNumber{-3, 4});
  assert(z.value() == 5.0);

  return 0;
}
```